รายงานการวิเคราะห์และออกแบบคลังข้อมูล

Data Warehouse Analysis and Design

ชื่อโครงการ: ระบบห่วงโซ่อุปทานการเกษตร (AgriChain Data Warehouse)

สถาปัตยกรรมหลัก: AWS Serverless, Amazon RDS for PostgreSQL, EventBridge, Step Functions, Lambda, ALB, EC2 และ DynamoDB

ตารางที่ 1 ข้อมูลควบคุมเอกสาร

รายการ	รายละเอียด
รหัสเอกสาร	DW-AGRI-001
ชื่อโครงการ	ระบบห่วงโซ่อุปทานการเกษตร (Agriculture Supply Chain)
หน่วยงานเจ้าของโครงการ	คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
ผู้ใช้งานเป้าหมาย	ฝ่ายบริหาร, ฝ่ายรับซื้อและบริหารคลังสินค้า, ฝ่ายขายและการตลาด, และฝ่ายจัดส่งและโลจิสติกส์
ผู้จัดทำ	นางสาวบุษยมาศ สนบ้านเกาะ, นางสาวพัชณกัญญ์ คชาวุธากร, นางสาวภารดี พิมลเอกอักษร, นางสาวยานวี ดูเบย์, นางสาววราภรณ์ สร้างคำ, นางสาวศุภิสรา หลายชั้น
สถานะเอกสาร	ฉบับสมบูรณ์

1. ความต้องการทางธุรกิจและขอบเขตโครงการ

1.1 วัตถุประสงค์และผู้ใช้ข้อมูล

โครงการจัดทำขึ้นเพื่อแก้ปัญหาการจัดเก็บข้อมูลแบบแยกส่วน (Data Silos) ของสหกรณ์การเกษตร โดยมุ่งสร้างระบบคลังข้อมูล (Data Warehouse) บนโครงสร้างพื้นฐานคลาวด์ให้เป็นแหล่งความจริงเพียงหนึ่งเดียว (Single Source of Truth) เพื่อให้ผู้ใช้งานแต่ละฝ่ายสามารถเข้าถึงและใช้ประโยชน์จากข้อมูลได้ดังนี้

- **ฝ่ายบริหาร :** วิเคราะห์ภาพรวมของธุรกิจ ประเมินประสิทธิภาพการดำเนินงาน และเปรียบเทียบสัดส่วนปริมาณผลผลิตที่รับเข้ากับความสามารถในการระบายสินค้า
- **ฝ่ายรับซื้อและบริหารคลังสินค้า :** ติดตามปริมาณผลผลิตที่เข้าสู่ระบบ วิเคราะห์อัตราการใช้พื้นที่คลังสินค้า และควบคุมการหมุนเวียนของผลผลิต
- **ฝ่ายขายและการตลาด :** วิเคราะห์พฤติกรรมการสั่งซื้อ ประเมินประสิทธิภาพช่องทางการขาย และจัดกลุ่มลูกค้าเพื่อวางแผนราคาและส่วนลด

- **ฝ่ายจัดส่งและโลจิสติกส์ :** ติดตามสถานะการส่งมอบ วิเคราะห์ต้นทุนค่าขนส่ง และประเมินประสิทธิภาพของรูปแบบการขนส่ง

1.2 ความต้องการทางธุรกิจ (Business Requirements)

ตารางที่ 2 Subject Area, Business Process และ KPI

ตัวชี้วัดหลักของระบบต้องมีนิยามเดียวกันทั้งใน Dashboard, AI Agent และรายงานสรุป เพื่อป้องกันการตีความตัวเลขไม่ตรงกันระหว่างฝ่ายธุรกิจและฝ่ายเทคนิค

Subject Area	Business Process	KPI	Dimension
การรับซื้อผลผลิต (Harvest)	การรับซื้อผลผลิตจากเกษตรกรเข้าสู่คลังสินค้า	- Total Harvest Amount (ยอดเงินรับซื้อรวม) - Total Quantity Harvested (ปริมาณรับซื้อรวม) - Average Price per Kg (ราคารับซื้อเฉลี่ย)	crop, warehouse, farmer, date
การขายผลผลิต (Sales)	การขายผลผลิตให้คู่ค้าและลูกค้า	- Total Sales Amount (ยอดขายรวม) - Total Quantity Sold (ปริมาณขายรวม) - Average Discount (ส่วนลดเฉลี่ย)	crop, warehouse, customer, date
การจัดส่ง (Shipment)	การขนส่งผลผลิตและติดตามสถานะส่งมอบ	- Total Shipping Cost (ค่าขนส่งรวม) - Average Delivery Time (ระยะเวลาจัดส่งเฉลี่ย) - Delayed Shipment Count (จำนวนจัดส่งล่าช้า)	crop, warehouse, customer, date

Subject Area	Business Process	KPI	Dimension
การจัดการสินค้าคงคลัง (Inventory)	การบันทึกและตรวจสอบสถานะผลผลิตคงเหลือในแต่ละคลัง	- Total Ending Stock (ปริมาณสต็อกคงเหลือ) - Inventory Turnover (อัตราหมุนเวียนสินค้า) - Stock Level Alert (จำนวนสินค้าที่ระดับสต็อกต่ำ)	crop, warehouse, date

1.3 นิยามตัวชี้วัด (KPI Definitions)

ตารางที่ 3 นิยามตัวชี้วัดหลักของระบบ

KPI	สูตร/แหล่งข้อมูล	หน่วยและเงื่อนไข
Total Harvest Amount	SUM(fact_harvest.total_amount_thb)	บาท; ใช้วันที่รับซื้อจาก dim_date
Total Sales Amount	SUM(fact_sales.total_amount_thb)	บาท; ควรหักส่วนลดตาม discount_pct หากใช้ยอดสุทธิ
Total Quantity Sold	SUM(fact_sales.quantity_kg)	กิโลกรัม; แยกได้ตาม crop, customer, warehouse และ date
Shipping Cost	SUM(fact_shipment.shipping_cost_thb)	บาท; ใช้เฉพาะรายการจัดส่งที่ผ่าน validation
Inventory Balance	SUM(fact_inventory.ending_balance_kg)	กิโลกรัม; grain คือรายวันต่อคลังสินค้าและผลผลิต

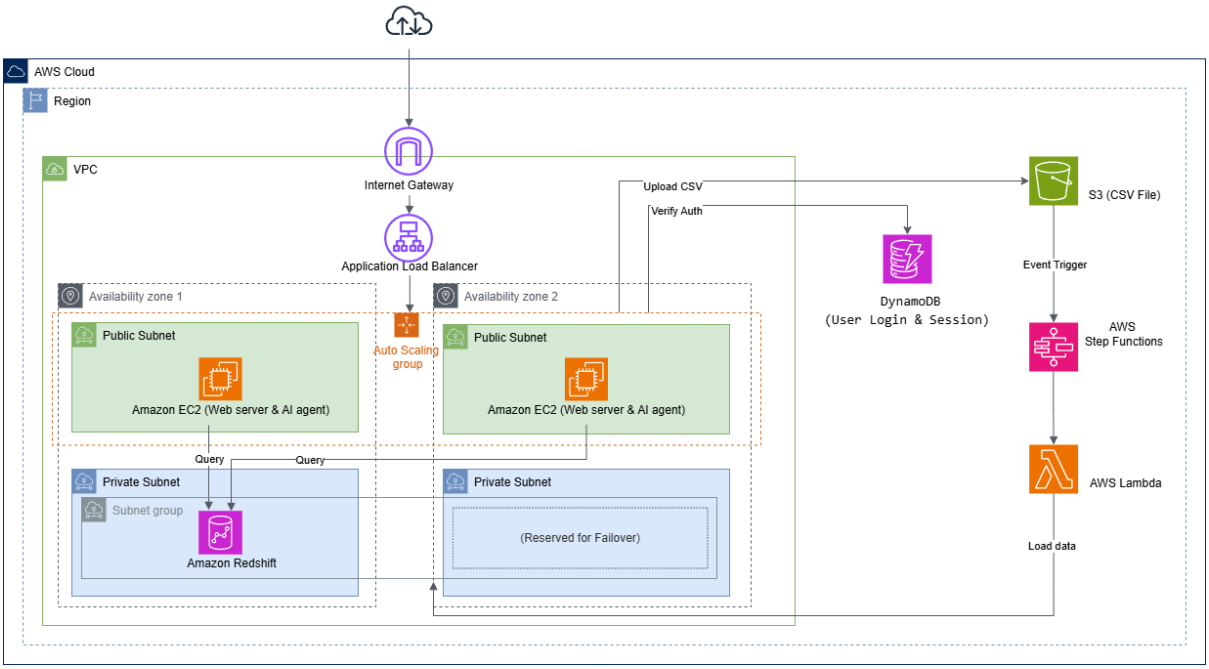
1.4 ขอบเขตและผลส่งมอบ

- **แหล่งข้อมูลต้นทาง :** รวบรวมข้อมูลปฏิบัติการรูปแบบ CSV จำนวน 8 ระบบ ได้แก่ ข้อมูลการรับซื้อ, การขาย, การจัดส่ง, สินค้าคงคลัง, เกษตรกร, ลูกค้า, ผลผลิต, และคลังสินค้า
- **โครงสร้างคลังข้อมูล :** จัดเก็บข้อมูลบนระบบฐานข้อมูล Amazon RDS ในรูปแบบจำลองมิติ (Star Schema)
- **กระบวนการจัดการข้อมูล (ETL) :** พัฒนากระบวนการดึง ทำความสะอาด และแปลงข้อมูลแบบอัตโนมัติด้วยสถาปัตยกรรมไร้เซิร์ฟเวอร์ ได้แก่ AWS Step Functions และ AWS Lambda
- **การแสดงผลและการวิเคราะห์ :** จัดทำชุดข้อมูลเพื่อแสดงผลผ่าน Dashboard และบูรณาการร่วมกับระบบปัญญาประดิษฐ์ (AI Agent) ที่รองรับการประมวลผลภาษาธรรมชาติแบบ Text-to-SQL

2. สถาปัตยกรรมและระบบต้นทาง

ระบบคลังข้อมูล AgriChain ถูกออกแบบบน Amazon Web Services (AWS) เพื่อรองรับการรับข้อมูลอัตโนมัติ การประมวลผลแบบไร้เซิร์ฟเวอร์ และการให้บริการผ่านเว็บแอปพลิเคชัน โดยระบบต้นแบบที่นำไปใช้งานจริงใช้ Region us-east-1 ภายใต้ข้อจำกัดของ AWS Learner Lab: ALB และ EC2 กระจาย 2 Availability Zones ส่วนฐานข้อมูล Amazon RDS for PostgreSQL ใช้แบบ Single-AZ เพื่อควบคุมต้นทุนและข้อจำกัดบัญชีทดลอง

2.1 สถาปัตยกรรมคลาวด์ที่นำไปใช้งานจริง (Implemented Cloud Architecture)



รูปที่ 1 AWS Architecture Design

2.2 ภาพรวมสถาปัตยกรรมและโครงสร้างเครือข่าย (Cloud Architecture & Security)

ระบบ AgriChain ทำงานบน Amazon Web Services (AWS) ใน Region us-east-1 โดยออกแบบเครือข่ายภายใน Amazon VPC ให้แยก Public Subnet สำหรับ ALB และ EC2 ออกจาก Private Subnet สำหรับ Amazon RDS และ AWS Lambda แนวคิดนี้ยังคงรองรับการขยายสู่ Multi-AZ เต็มรูปแบบใน production แม้ระบบต้นแบบจะลดบางส่วนตามข้อจำกัดของ Learner Lab

- **Public Subnet (Multi-AZ) :** เชื่อมต่ออินเทอร์เน็ตผ่าน Internet Gateway สำหรับวาง Application Load Balancer (ALB) และเครื่อง Amazon EC2 (Web Server & AI Agent) อนุญาต Ingress เฉพาะ Port 80 (HTTP), Port 3000 (Web App) และ Port 7860 (Gradio API)

- **Private Subnet (Multi-AZ) :** พื้นที่ปิดการเข้าถึงจากอินเทอร์เน็ตโดยตรง ใช้สำหรับคลังข้อมูล Amazon RDS (PostgreSQL) และกระบวนการ ETL (AWS Lambda) โดยจำกัด Ingress Port 5432 เฉพาะ Security Group ของ EC2 และ Lambda เท่านั้น

2.3 โครงสร้างการไหลของข้อมูล (Data Flow Architecture)

ลำดับขั้นตอนการประมวลผลและการทำงานของข้อมูลแบ่งออกเป็น 3 ส่วนหลัก

1. **Automated ETL Pipeline :** ผู้ดูแลระบบอัปโหลดไฟล์ CSV เข้าสู่ Amazon S3 (s3://agri-data-lake-raw/) ส่งสัญญาณ Event Trigger ผ่าน Amazon EventBridge ไปยัง AWS Step Functions เพื่อสั่งการ AWS Lambda ให้ดึง ทำความสะอาด แปลงข้อมูล และทำ Dynamic Upsert (ON CONFLICT) เข้า Amazon RDS PostgreSQL หากเกิดข้อผิดพลาด Step Functions จะ Retry อัตโนมัติ 3 ครั้ง หากไม่สำเร็จจะย้ายไฟล์ไป Quarantine Bucket เพื่อตรวจผ่าน CloudWatch Logs
2. **Machine Learning Prediction :** เซิร์ฟเวอร์ EC2 ดึงข้อมูลประวัติจาก Amazon RDS ไปประมวลผลผ่านโมเดล Machine Learning เพื่อทำนายแนวโน้มทางธุรกิจ แล้วบันทึกผลลัพธ์พร้อมค่าความแม่นยำกลับลง RDS ใน Analytics Schema
3. **Visualization & AI Agent Integration :** ผู้ใช้งานเข้าสู่ระบบผ่าน Web Application เพื่อดูตัวชี้วัดบน Dashboard หรือโต้ตอบผ่าน AI Agent (Text-to-SQL) โดยมี SQL Guardrails ควบคุมสิทธิ์เป็น Read-Only (SELECT), จำกัด Schema, บังคับ LIMIT 100 และตั้งค่า Statement Timeout ไว้ที่ 15 วินาที

2.4 องค์ประกอบระบบ (System Components)

ตารางที่ 4 องค์ประกอบและค่าตั้งค่าสำคัญของระบบ (AWS System Components)

องค์ประกอบ	หน้าที่	Configuration และ Security สำคัญ
Amazon S3	Landing Zone เก็บไฟล์ CSV ดิบเท่านั้น	ตั้งค่า Block all public access และใช้ Event Notification กระตุ้นกระบวนการ ETL ทันทีที่มีไฟล์ใหม่
AWS Lambda & Step Functions	Serverless ETL & Pipeline Orchestrator	รัน Python 3.12 (RAM 2,048 MB), อยู่ใน Private Subnet เชื่อมต่อ S3 ผ่าน Gateway VPC Endpoint, Timeout สูงสุด 900 วินาที
Amazon RDS (PostgreSQL)	คลังข้อมูลระดับ OLAP (Star Schema)	PostgreSQL 16.3 (ขนาด db.t3.micro), อยู่ใน Private Subnet, Publicly accessible = false, เปิดรับ Port 5432 เฉพาะจาก EC2 และ Lambda

องค์ประกอบ	หน้าที่	Configuration และ Security สำคัญ
Amazon EC2 & Auto Scaling	รัน Web Backend (Port 3000) และ AI Agent (Port 8000)	Target Tracking Policy ปรับขนาดเมื่อ CPU > 60% (เริ่มต้น 1 เครื่อง สูงสุด 4 เครื่อง)
Application Load Balancer	กระจายทราฟฟิกข้าม 2 AZs	Internet-facing, รับ HTTP Port 80, Health check ที่ /healthz, เปิด Sticky session (lb_cookie) อายุ 86,400 วินาที
Amazon DynamoDB	จัดเก็บข้อมูลแบบ NoSQL	เก็บข้อมูลบัญชีผู้ใช้, User Session (กำหนดค่า TTL หมดอายุ) และประวัติแชท เพื่อให้ EC2 ทำงานแบบ Stateless

2.5 รายการระบบต้นทางและรอบรับข้อมูล (Source Systems)

ข้อมูลปฏิบัติการถูกเก็บรวบรวมจากแผนกต่าง ๆ ของสหกรณ์การเกษตรในรูปแบบไฟล์ CSV ทั้งหมด 8 ระบบ ดังนี้

ตารางที่ 5 รายการระบบต้นทาง (Source Systems)

แหล่งข้อมูล (ตาราง)	รายละเอียด	คอลัมน์หลัก (สำคัญ)
Harvest.csv	ข้อมูลการรับซื้อผลผลิตจากเกษตรกรเข้าสู่คลัง	harvest_id, farmer_id, crop_id, warehouse_id, harvest_date, quantity_kg, price_per_kg
Sales.csv	ข้อมูลการขายผลผลิตและสินค้าให้กับลูกค้า	sales_id, order_id, customer_id, crop_id, sale_date, quantity_kg, total_amount_thb
Shipment.csv	ข้อมูลการจัดส่งสินค้าไปยังลูกค้า	shipment_id, warehouse_id, customer_id, delivery_date, status, transport_mode
Inventory.csv	ข้อมูลบันทึกสถานะสินค้าคงคลังรายรอบ	inventory_id, warehouse_id, crop_id, snapshot_date, beginning_stock_kg, ending_stock_kg

แหล่งข้อมูล (ตาราง)	รายละเอียด	คอลัมน์หลัก (สำคัญ)
Farmer.csv	ข้อมูลทะเบียนประวัติเกษตรกร (Master Data)	farmer_id, farmer_name, province, cooperative, primary_crop, registration_date
Customer.csv	ข้อมูลประวัติลูกค้าและคู่ค้า (Master Data)	customer_id, customer_name, customer_type, region, credit_limit_thb
Crop.csv	ข้อมูลรายละเอียดผลผลิต (Master Data)	crop_id, crop_name, category, standard_price_per_unit, season_months
Warehouse.csv	ข้อมูลคลังสินค้าและจุดรับซื้อ (Master Data)	warehouse_id, warehouse_name, region, capacity_ton, current_utilization_pct

3. แบบจำลองข้อมูลและโครงสร้างคลังข้อมูล

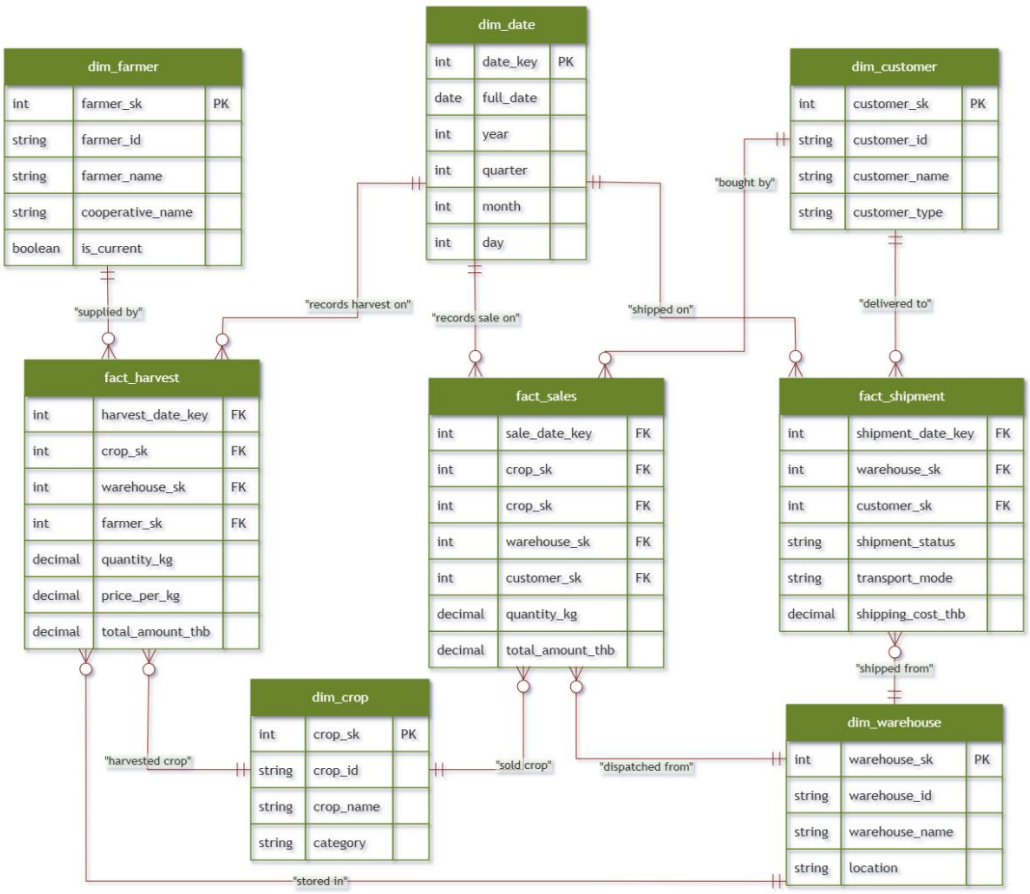
3.1 Enterprise Data Warehouse Bus Matrix

การออกแบบคลังข้อมูลได้อ้างอิงความสัมพันธ์ระหว่างกระบวนการทางธุรกิจ (Business Process) และมิติข้อมูลร่วม (Conformed Dimensions) เพื่อให้เกิดศูนย์กลางความจริงเพียงแหล่งเดียว (Single Source of Truth) ดังตารางด้านล่าง

ตารางที่ 6 Business Process และมิติร่วม (Bus Matrix)

Business Process	Crop	Warehouse	Farmer	Customer	Date
การรับซื้อผลผลิต (Harvest)	✓	✓	✓		✓
การขายผลผลิต (Sales)	✓	✓		✓	✓
การจัดส่ง (Shipment)	✓	✓		✓	✓
การจัดการสินค้าคงคลัง (Inventory)	✓	✓			✓

3.2 แบบจำลองข้อมูล (Entity Relationship Diagram - Star Schema)



รูปที่ 2 Entity Relationship Diagram

คลังข้อมูลถูกออกแบบในรูปแบบแบบจำลองมิติ (Star Schema) บน Amazon RDS โดยลดความซ้ำซ้อนของตารางมิติ (Denormalized) เพื่อเพิ่มประสิทธิภาพในการสืบค้นข้อมูลเชิงวิเคราะห์และดึงข้อมูลไปแสดงผลบน Dashboard ประกอบด้วย

- **ตารางข้อเท็จจริง (Fact Tables) 4 ตาราง :** fact_harvest (รับซื้อ), fact_sales (ยอดขาย), fact_shipment (จัดส่ง), และ fact_inventory (คงคลัง)
- **ตารางมิติ (Dimension Tables) 5 ตาราง :** dim_crop (ผลผลิต), dim_warehouse (คลังสินค้า), dim_farmer (เกษตรกร), dim_customer (ลูกค้า), และ dim_date (วันที่)

3.3 ข้อกำหนดการจัดการประวัติ (Slowly Changing Dimension - SCD)

เพื่อให้คลังข้อมูลรองรับการเปลี่ยนแปลงของข้อมูลจากระบบปฏิบัติการอย่างถูกต้อง จึงกำหนดนโยบาย SCD ในแต่ละตารางมิติดังนี้

ตารางที่ 7 นโยบายประวัติรายมิติ (SCD Policies)

มิติ (Dimension)	นโยบาย (SCD Type)	กลไกการจัดการประวัติ (Historical Tracking)
dim_crop	Type 1	อัปเดตค่าทับข้อมูลเดิม (Overwrite) เนื่องจากเป็นการแก้ไขชื่อหรือปรับปรุงข้อมูลมาตรฐาน
dim_farmer	Type 2	เพิ่มแถวใหม่ (Insert Row) พร้อมระบุ valid_from, valid_to, และ is_current เพื่อติดตามการย้ายสหกรณ์หรือเปลี่ยนขนาดพื้นที่
dim_customer	Type 2	เพิ่มแถวใหม่ พร้อมระบุ valid_from, valid_to, และ is_current เพื่อติดตามการเปลี่ยนที่อยู่หรือการเพิ่ม/ลดวงเงินเครดิต
dim_warehouse	Type 3	เก็บชื่อเดิมใน previous_warehouse_name และอัปเดตชื่อใหม่ลง current_warehouse_name
dim_date	Type 0	ข้อมูลปฏิทินเป็นค่าคงที่ โหลดครั้งเดียว (Insert Only) และห้ามแก้ไขหรืออัปเดตค่า

3.4 พจนานุกรมข้อมูลและทะเบียนตาราง (Data Dictionary and Table Registry)

ตารางอธิบายรายละเอียดพจนานุกรมข้อมูล (Data Dictionary) ของระบบ Data Warehouse ซึ่งประกอบด้วยตารางมิติ (Dimension Tables) และตารางข้อเท็จจริง (Fact Tables) ที่ได้รับการปรับปรุงโครงสร้างจากข้อมูลดิบแล้ว โดยมีการใช้ Surrogate Key (SK) เพื่อเพิ่มประสิทธิภาพในการเชื่อมโยงข้อมูล

3.5 โครงสร้างตารางข้อเท็จจริง (Fact Table Specifications)

ตารางข้อเท็จจริงกำหนดระดับความละเอียด (grain) ให้ชัดเจนเพื่อควบคุมการนับซ้ำ การ join กับมิติ และการคำนวณ KPI ใน Dashboard และ AI Agent

ตารางที่ 8 แสดงตารางข้อเท็จจริง (Fact Tables)

Table Name	Description	Granularity	Key Columns	Related Dimensions
fact_harvest	เก็บข้อมูลการรับซื้อผลผลิตทางเกษตรเข้าสู่คลัง	1 แถว = 1 รายการรับซื้อ	harvest_id	dim_date, dim_farmer, dim_crop, dim_warehouse
fact_sales	เก็บข้อมูลการขายผลผลิตให้กับลูกค้า/คู่ค้า	1 แถว = 1 รายการขาย	sales_id	dim_date, dim_customer, dim_crop, dim_warehouse
fact_shipment	เก็บข้อมูลการขนส่งผลผลิตไปยังลูกค้า	1 แถว = 1 รอบการจัดส่ง	shipment_id	dim_date (shipment_date_key, delivery_date_key), dim_customer, dim_crop, dim_warehouse
fact_inventory	เก็บข้อมูลสถานะสินค้าคงคลังรายรอบ	1 แถว = 1 บันทึกยอดคงเหลือต่อคลัง	inventory_id	dim_date (snapshot_date_key), dim_warehouse, dim_crop

3.6 โครงสร้างตารางมิติ (Dimension Table Specifications)

ตารางมิติใช้ business key จากระบบต้นทางร่วมกับ surrogate key ในคลังข้อมูล เพื่อให้รองรับการเปลี่ยนแปลงข้อมูลตามนโยบาย SCD และช่วยให้ fact table อ้างอิงข้อมูลได้เสถียร

ตารางที่ 9 แสดงตารางมิติ (Dimension Tables)

Dimension	Description	Key	Attributes
dim_crop	รายละเอียดผลผลิตทางการเกษตร	crop_sk	crop_id, crop_name, category, unit, standard_price_per_unit, season_months, shelf_life_days
dim_warehouse	รายละเอียดคลังสินค้าและไซโล (Silo)	warehouse_sk	warehouse_id, warehouse_name, province, region, capacity_ton, manager_name

Dimension	Description	Key	Attributes
dim_farmer	ข้อมูลประวัติเกษตรกรผู้ส่งมอบผลผลิต	farmer_sk	farmer_id, farmer_name, province, region, cooperative, farm_size_rai, primary_crop, valid_from, valid_to, is_current
dim_customer	ข้อมูลลูกค้าและคู่ค้าขอสหกรณ์	customer_sk	customer_id, customer_name, customer_type, province, region, credit_limit_thb, valid_from, valid_to, is_current
dim_date	ปฏิทินเวลามาตรฐาน	date_key	date, day, month, quarter, year, weekday

4. กระบวนการรับ แปลง และโหลดข้อมูล (ETL Processes)

กระบวนการจัดการข้อมูลของระบบ AgriChain ถูกออกแบบให้ทำงานแบบอัตโนมัติ (Automated Data Pipeline) บนสถาปัตยกรรมไร้เซิร์ฟเวอร์ (Serverless) ของ AWS เพื่อลดภาระของผู้ดูแลระบบและเพิ่มความรวดเร็วในการประมวลผลข้อมูล

4.1 สรุปกระบวนการ ETL (Extract, Transform, Load)

กระบวนการ ETL เป็นแบบไร้เซิร์ฟเวอร์อัตโนมัติ เริ่มจากการสกัดข้อมูล (Extract) เมื่อไฟล์ CSV ถูกอัปโหลดลง S3 แล้วส่งสัญญาณ Event ไปยัง Step Functions และ Lambda จากนั้นทำการแปลงข้อมูล (Transform) โดยตรวจสอบกฎคุณภาพข้อมูล จัดการค่าว่าง และแปลงรูปแบบวันที่ ก่อนจะโหลดข้อมูล (Load) เข้าสู่ Amazon RDS โดยจะประมวลผลตารางมิติก่อนเพื่อสร้าง Surrogate Key (SK) แล้วจึงโหลดตารางข้อเท็จจริงพร้อมเชื่อมโยง FK ด้วยเทคนิค Bulk Upsert (ON CONFLICT)

4.2 รายละเอียดการประมวลผลและกฎคุณภาพข้อมูล

- **การทำความสะอาดข้อมูล (Data Cleansing):**
    - ตัดช่องว่างข้อความ (Trim)
    - จัดการค่าว่าง: อนุญาตให้ delivery_date เป็น Null ได้หากยังไม่จัดส่ง แต่ค่าคำนวณ เช่น quantity_kg, discount_pct, received_kg หากว่างจะถูกแปลงเป็น 0 หรือ 0.00 อัตโนมัติ

    - ตรวจสอบความถูกต้อง: ปริมาณและจำนวนเงินต้อง ≥ 0 หากติดลบจะถูกปฏิเสธ (Reject) และตรวจสอบให้ delivery_date ไม่เกิดก่อน shipment_date
- **การสร้าง Surrogate Key (SK):**
    - ตารางมิติจะสร้างคีย์ตัวแทนประเภท Integer อัตโนมัติ (เช่น crop_sk, farmer_sk)
    - ตาราง dim_date แปลงวันที่รูปแบบ YYYY-MM-DD เป็น Integer 8 หลัก (YYYYMMDD) กำหนดเป็น date_key
    - ตารางข้อเท็จจริงจะทำ SK Lookup ข้ามตารางมิติ โดยตารางที่มี SCD Type 2 จะ Lookup SK เฉพาะแถวที่ is_current = true
- **การจัดการข้อมูลเปลี่ยนตามเวลา (SCD Logics):**
    - dim_crop (SCD Type 1): อัปเดตทับข้อมูลเดิม
    - dim_farmer และ dim_customer (SCD Type 2): เพิ่มแถวใหม่ พร้อมกำหนด valid_from (วันปัจจุบัน) และ valid_to (9999-12-31) เพื่อเก็บประวัติ
    - dim_warehouse (SCD Type 3): ย้ายชื่อเดิมไป previous_warehouse_name และบันทึกชื่อใหม่ลง current_warehouse_name
- **การทำ Upsert:** ใช้คำสั่ง INSERT INTO ... ON CONFLICT (primary_key) DO UPDATE SET ... สำหรับตารางข้อเท็จจริง เพื่อป้องกันการบันทึกข้อมูลซ้ำซ้อน

4.3 กฎการตรวจสอบและทำความสะอาดข้อมูล (Data Cleansing & Validation Rules)

ข้อมูลดิบจากไฟล์ CSV จะต้องผ่านกฎการตรวจสอบดังต่อไปนี้ก่อนเข้าสู่คลังข้อมูล

ตารางที่ 10 เงื่อนไขคุณภาพข้อมูล (Data Quality Rules)

รหัสและการตรวจ	เกณฑ์การจัดการและการแปลงค่า (Transformation Logic)
DQ01 ค่าว่าง (Null/Missing)	- delivery_date หากสถานะยังไม่จัดส่ง อนุญาตให้เป็น Null ได้ แต่ถ้าสถานะ "จัดส่งแล้ว" ต้องมีวันที่เสมอ และค่าว่างในคอลัมน์คำนวณ เช่น quantity_kg, discount_pct, received_kg จะถูกแปลงเป็น 0 อัตโนมัติเพื่อป้องกัน Error
DQ02 มาตรฐานรูปแบบ (Format)	- วันที่ทั้งหมดถูกแปลงเป็นมาตรฐาน YYYY-MM-DD เพื่อสร้าง date_key (เช่น 20250428) และข้อมูลประเภทข้อความ (เช่น farmer_name, customer_name) ต้องผ่านการลบช่องว่างหน้า/หลัง (Trim)

DQ03 ความถูกต้องเชิงตรรกะ	- ปริมาณผลผลิตและจำนวนเงิน (quantity_kg, price_per_kg, total_amount_thb) ต้อง >= 0 เสมอ หากติดลบจะถูกคัดแยกออก (Reject) และ delivery_date ต้องไม่เกิดก่อน shipment_date

4.4 Source to Target Mapping (ตารางมิติ)

ตารางที่ 11 Mapping ของตารางมิติ (Dimension Tables)

ปลายทาง (Target)	แหล่งข้อมูล (Source CSV)	การแปลงและการโหลด (Transformation & Load Logic)
dim_crop	Crop.csv	สร้าง crop_sk อัตโนมัติ (PK) โหลดข้อมูล crop_id, crop_name, category, unit, season_months, shelf_life_days โดยตรง หาก standard_price_per_unit เป็น Null ให้แปลงเป็น 0.00 (SCD Type 1)
dim_farmer	Farmer.csv	สร้าง farmer_sk อัตโนมัติ (PK) ดึง farmer_id, farmer_name, farm_size_rai (ห้าม Null), province, primary_crop สร้าง valid_from (วันปัจจุบัน) และ valid_to (9999-12-31) เพื่อจัดการ SCD Type 2 หากชื่อสหกรณ์ (cooperative) เปลี่ยน
dim_warehouse	Warehouse.csv	สร้าง warehouse_sk อัตโนมัติ (PK) ดึง warehouse_id, region, capacity_ton (ห้าม Null/ติดลบ) หากคลังเปลี่ยนชื่อ จะย้ายชื่อเดิมไป previous_warehouse_name และรับค่าใหม่ลง current_warehouse_name (SCD Type 3)
dim_customer	Customer.csv	สร้าง customer_sk อัตโนมัติ (PK) แปลงค่าว่างใน credit_limit_thb เป็น 0.00 สร้าง valid_from และ valid_to (9999-12-31) สำหรับจัดการ SCD Type 2
dim_date	Harvest.csv, Sales.csv, Shipment.csv	สกัดจากคอลัมน์วันที่ แปลงรูปแบบเป็นตัวเลข 8 หลัก (YYYYMMDD) ลง date_key (PK) และสกัด Month, Quarter, Year ออกมาบันทึกแยก

4.5 Source to Target Mapping (ตารางข้อเท็จจริง)

ตารางที่ 12 Mapping ของตารางข้อเท็จจริง (Fact Tables)

ปลายทาง (Target)	แหล่งข้อมูล (Source CSV)	การเชื่อมโยงคีย์และการคำนวณ (Lookup & Logic)
fact_harvest	Harvest.csv	ใช้ harvest_id เป็น PK แปลงวันที่รับซื้อเพื่อ Lookup FK date_key นำรหัสไป Lookup FK farmer_sk (ณ ช่วงเวลา is_current = true), crop_sk, warehouse_sk คำนวณความถูกต้องซ้ำ total_amount_thb = quantity_kg * price_per_kg
fact_sales	Sales.csv	ใช้ sales_id เป็น PK นำรหัสไป Lookup FK customer_sk, crop_sk, warehouse_sk บังคับ quantity_kg และ unit_price_thb ห้ามติดลบ หาก discount_pct เป็น Null ให้แปลงเป็น 0
fact_shipment	Shipment.csv	ใช้ shipment_id เป็น PK แปลงวันที่เพื่อ Lookup FK shipment_date_key ส่วน delivery_date หากเป็น Null (ยังไม่ส่ง) ให้ Map FK เป็น 99991231 Lookup FK ข้ามตารางมิติลูกค้า สินค้า และคลังสินค้า บังคับค่าน้ำหนักและค่าจัดส่งห้ามติดลบ
fact_inventory	Inventory.csv	ใช้ inventory_id เป็น PK Lookup FK snapshot_date_key, warehouse_sk, crop_sk หาก received_kg และ sold_kg เป็น Null ให้แปลงเป็น 0 บังคับยอดยกมา ยอดสิ้นรอบ และต้นทุนห้ามติดลบ

4.6 Data Lineage และการกู้คืนกระบวนการโหลด

เส้นทางข้อมูลของระบบคือ CSV จากผู้ดูแลระบบถูกจัดเก็บใน S3 Raw Zone จากนั้น EventBridge กระตุ้น Step Functions เพื่อเรียก Lambda ทำ validation, cleansing, surrogate key lookup และ upsert ลง Amazon RDS หาก batch ล้มเหลวต้องบันทึกสถานะไฟล์ ชื่อตาราง จำนวนแถวที่รับเข้า จำนวนแถวที่ reject และข้อความผิดพลาด เพื่อให้สามารถแก้ไฟล์ต้นทางและ re-run เฉพาะ batch เดิมได้โดยไม่กระทบข้อมูลที่เผยแพร่แล้ว

5. การแสดงผลและการประยุกต์ใช้ปัญญาประดิษฐ์ (Visualization & AI Agent)

5.1 หน้าเข้าสู่ระบบ (Login)

ผู้ใช้กรอกชื่อผู้ใช้และรหัสผ่านเพื่อยืนยันตัวตนกับตาราง agrichain_user_login ใน DynamoDB ผ่านการแฮช ด้วย scrypt เมื่อผ่านการตรวจสอบ ระบบจะสร้าง Session Token ที่มี TTL หมดอายุอัตโนมัติจัดเก็บลง DynamoDB และส่ง Cookie กลับไปยังเบราว์เซอร์ เพื่อใช้ยืนยันสิทธิ์ในทุกคำขอ /api/* แบบ Stateless



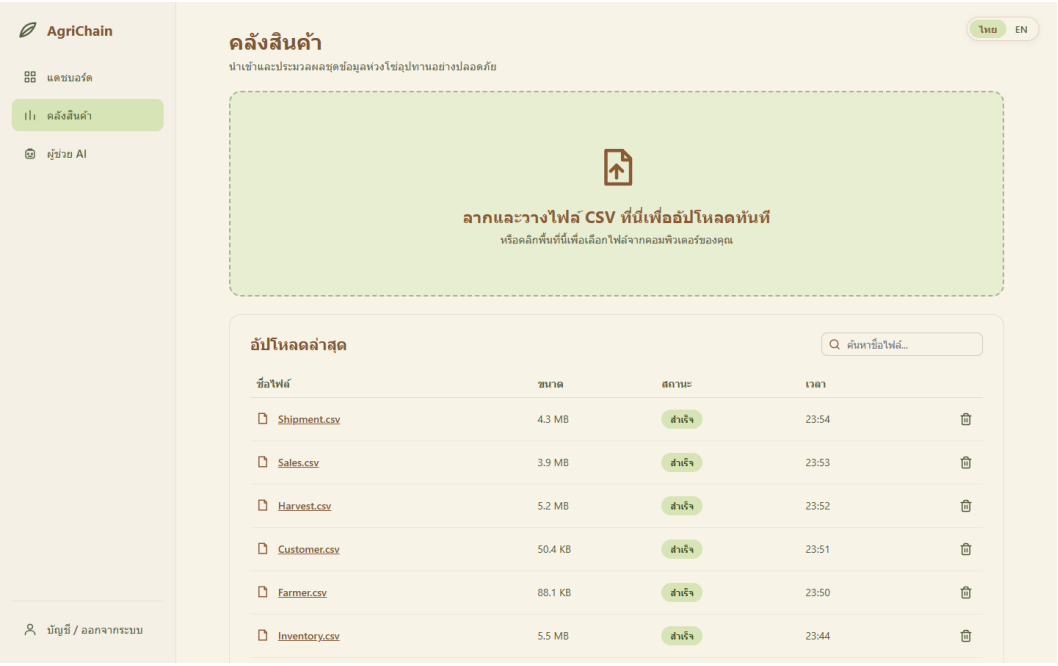
รูปที่ 3 หน้าเข้าสู่ระบบ (Login)



รูปที่ 4 หน้าเข้าสู่ระบบเมื่อกรอกชื่อผู้ใช้และรหัสผ่าน

5.2 หน้าคลังสินค้า (นำเข้าข้อมูล CSV)

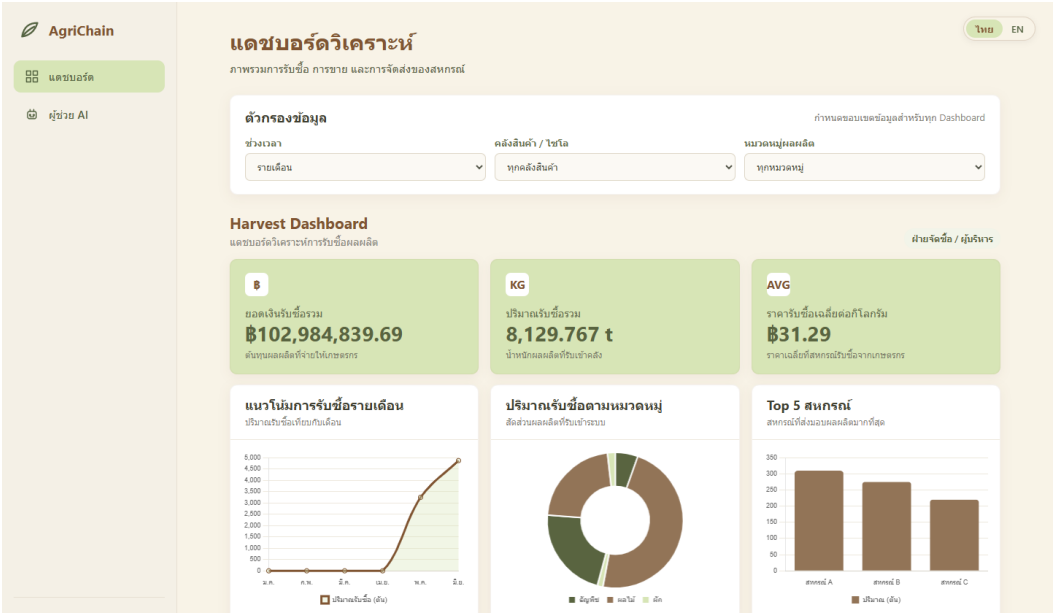
ผู้ดูแลระบบสามารถลากและวางหรือเลือกไฟล์ CSV เพื่ออัปโหลดเข้าสู่ Amazon S3 (csv-uploads/) โดยหน้าจอจะแสดงรายการไฟล์ล่าสุด ขนาด สถานะ และเวลาอัปโหลด การอัปโหลดไฟล์ที่มีคำสำคัญตรงกับตาราง (เช่น Crop, Farmer, Sales) จะกระตุ้น Event-Driven ETL Pipeline บน AWS ให้ประมวลผลเข้าฐานข้อมูลโดยอัตโนมัติ



รูปที่ 5 หน้าคลังสินค้า : อัปโหลดไฟล์ CSV และรายการไฟล์ที่นำเข้าแล้ว 8 ไฟล์

5.3 Dashboard

หน้าจอแสดงผลการวิเคราะห์ภาพรวมธุรกิจด้วยตัวชี้วัดหลัก (KPI Cards) และกราฟโต้ตอบจาก Chart.js ระบบดึงข้อมูลผ่าน API /api/dashboard ซึ่งประมวลผลการสรุปผลยอดรับซื้อ ยอดขาย สต็อกคงเหลือ และประสิทธิภาพการจัดส่งมาจาก Amazon RDS PostgreSQL โดยตรงตามตัวกรองที่ผู้ใช้เลือก



รูปที่ 6 Dashboard

5.4 หน้าผู้ช่วย AI (AI Assistant)

หน้าต่างสนทนาภาษาธรรมชาติที่ช่วยให้ผู้ใช้สืบค้นข้อมูลผ่านโมเดล **Qwen (ผ่าน Groq API)** เพื่อแปลงคำถามเป็นคำสั่ง **SQL (Text-to-SQL)** คำสั่งจะถูกส่งผ่าน **SQL Guardrail** เพื่อตรวจสอบความปลอดภัยก่อนรันบน **RDS** แล้วนำผลลัพธ์มาสรุปเป็นข้อความภาษาไทย กราฟวิเคราะห์ และแสดงปุ่มกดเปิดดูคำสั่ง **SQL** ที่ใช้ดึงข้อมูลได้



รูปที่ 7 หน้าผู้ช่วย AI : ตัวอย่างการถามยอดขายสุทธิ ยอดขายรวมตามพืช



รูปที่ 8 หน้าผู้ช่วย AI เมื่อเปิดดูคำสั่ง SQL ที่ระบบสร้างและใช้ดึงข้อมูล

6. การกำกับดูแล ความปลอดภัย และการปฏิบัติงาน (Governance, Security & Operations)

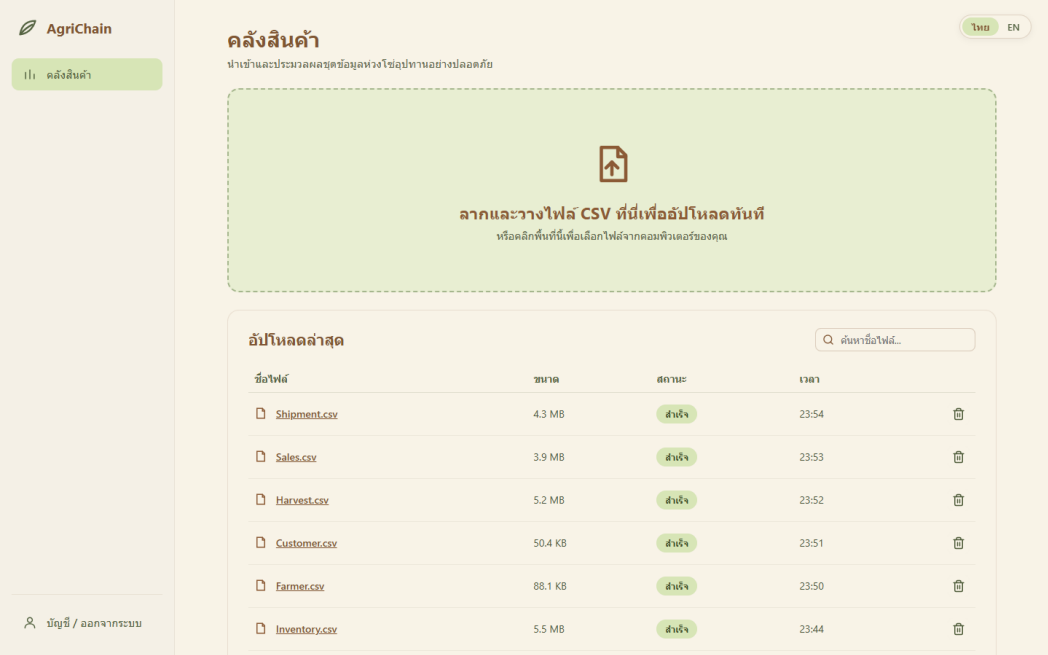
6.1 การควบคุมการเข้าถึง (Role-Based Access Control - RBAC)

ระบบมีการกำหนดสิทธิ์การเข้าถึงข้อมูลตามบทบาทผู้ใช้งาน 3 ระดับ ได้แก่ Data Admin (จัดการข้อมูลและระบบ), Data Analyst (ดูแดชบอร์ดเชิงลึก) และ Executive (ใช้งาน AI Agent และดูภาพรวม) การจัดการเซสชัน (Session) ของผู้ใช้จะถูกจัดเก็บและตรวจสอบความถูกต้องผ่าน Amazon DynamoDB เพื่อรองรับการทำงานแบบไร้สถานะ (Stateless) บนเซิร์ฟเวอร์

ตารางที่ 13 แสดงบทบาทผู้ใช้งาน 3 ระดับ

ผู้ใช้	บทบาท (role)	หน้าแรกหลังล็อกอิน	หน้าที่เข้าได้	ใช้งานตาม Use Case
admin01	Data Admin	คลังสินค้า (inventory.html)	คลังสินค้า	อัปโหลดชุดข้อมูล, ดูและลบไฟล์ที่นำเข้า
analyst01	Data Analyst	แดชบอร์ด (dashboard.html)	แดชบอร์ด, ผู้ช่วย AI	เรียกดู Dashboard, สอบถามข้อมูลด้วย AI Agent

ผู้ใช้	บทบาท (role)	หน้าแรกหลังล็อกอิน	หน้าที่เข้าได้	ใช้งานตาม Use Case
exec01	Executive	แดชบอร์ด (dashboard.html)	แดชบอร์ด, ผู้ช่วย AI	เรียกดู Dashboard, สอบถามข้อมูลด้วย AI Agent



รูปที่ 9 มุมมองของ Data Admin (admin01) : เห็นเมนูเฉพาะ "คลังสินค้า"



รูปที่ 10 มุมมองของ Data Analyst (analyst01) : เห็นเมนู "แดชบอร์ด" และ "ผู้ช่วย AI"

6.2 ความปลอดภัยของเครือข่าย (Network Security Groups)

การตั้งค่า Security Groups ถูกออกแบบมาเพื่อจำกัดการเข้าถึงให้แคบที่สุด (Least Privilege) โดย Application Load Balancer (ALB) จะเปิดรับพอร์ต 80 จากอินเทอร์เน็ตเท่านั้น เครื่องเซิร์ฟเวอร์ EC2 จะเปิดพอร์ต 3000 เพื่อรับทราฟฟิกจาก ALB เท่านั้น ส่วนฐานข้อมูล Amazon RDS จะเปิดรับพอร์ต 5432 เฉพาะจาก Security Group ของ EC2 และ AWS Lambda เพื่อป้องกันการเข้าถึงฐานข้อมูลจากภายนอกโดยตรง

6.3 ประมาณการค่าใช้จ่าย (Cost Management)

ตารางที่ 14 แสดงประมาณการค่าใช้จ่าย

AWS Service	Component Description / Capacity Specification	Estimated Monthly Cost (USD)
Amazon RDS	PostgreSQL (db.t4g.medium, Multi-AZ, 100 GB Storage)	~$115.00
Amazon EC2	t3.medium (Web App Dashboard & AI Agent API Node)	~$30.40
Application Load Balancer	1 ALB Instance + LCU Usage	~$22.50
Amazon S3	Data Lake Storage (Raw/Processed/Archive - ~50 GB)	~$1.15
AWS Lambda & Step Functions	ETL Ingestion Workflows (~10,000 executions/month)	~$2.50
Amazon CloudWatch	Logs, Metrics & Alarm Monitoring	~$5.00
Total Estimated Cost	ประมาณการค่าใช้จ่ายรวมต่อเดือน	~$176.55 / เดือน

หมายเหตุ : ตารางนี้เป็นประมาณการสำหรับสถาปัตยกรรมเป้าหมายที่เปิดบริการต่อเนื่อง ส่วนระบบต้นแบบใน Learner Lab ใช้ RDS Single-AZ, เปิด EC2/RDS เฉพาะช่วงทดสอบ และมีข้อจำกัดเรื่อง IAM Role, HTTPS และ Redshift ตามที่สรุปในหัวข้อข้อจำกัดของระบบ

6.4 แนวทางการบริหารจัดการค่าใช้จ่าย (Cost Optimization Strategies)

1. Auto-Scaling Policy: กำหนด Minimum Instance เป็น 1 Node ในช่วงนอกเวลาทำการ และ Scaling Up เมื่อ CPU Exceeds 70%
2. S3 Lifecycle Rules: ย้ายไฟล์ Raw Data ใน S3 ที่มีอายุเกิน 90 วัน ไปยัง S3 Glacier Flexible Retrieval เพื่อลดค่า Storage
3. RDS Non-Production Pause: ตั้งค่าปิดการทำงานของ Development Database Automatic Stop/Start ตามช่วงเวลาที่ไม่ได้รับส่งข้อมูล

7. การทดสอบและรายละเอียดการนำไปใช้งาน (Testing & Deployment)

7.1 เกณฑ์รับมอบระบบ (Acceptance Criteria)

ตารางที่ 15 เกณฑ์รับมอบระบบ

หมวด	เกณฑ์รับมอบ	หลักฐานที่ควรแนบ
ETL	โหลดไฟล์ CSV ทั้ง 8 ชุดได้ครบ และยอดรวมแถวตรงกับรายงานการโหลด	ตารางผลโหลด, CloudWatch logs, Step Functions execution
Data Quality	ข้อมูลที่ผิดรูปแบบถูก reject พร้อมเหตุผล และไม่มีค่า FK หลุดจาก dimension	ตาราง rejected rows และ validation summary
Dashboard	KPI หลักแสดงผลตรงกับ query จาก RDS	ภาพหน้าจอ Dashboard และ SQL ตรวจสอบ
AI Agent	สร้างเฉพาะ SQL แบบ SELECT, จำกัด schema, LIMIT และ timeout	คำถามทดสอบ, SQL ที่สร้าง, ผลลัพธ์
Operations	ปิด/เปิดระบบแล้วข้อมูลใน RDS, S3 และ DynamoDB ยังคงอยู่	ผลทดสอบ /healthz และขั้นตอน start/stop

7.2 ผลการนำเข้าข้อมูล (ETL)

ทดสอบโดยอัปโหลดไฟล์ CSV ทั้ง 8 ไฟล์ตามลำดับที่กำหนดผ่านหน้าคลังสินค้า ทุกไฟล์ผ่าน กระบวนการ Step Functions และ Lambda สำเร็จ โหลดข้อมูลรวม 170,504 แถวเข้าสู่ schema

ตารางที่ 16 ผลการนำเข้าไฟล์ CSV ด้วย ETL

ลำดับ	ไฟล์	ตารางปลายทาง	จำนวนแถวที่โหลด	ผล
1	Crop.csv	dim_crop	15	สำเร็จ
2	Farmer.csv	dim_farmer	300	สำเร็จ
3	Warehouse.csv	dim_warehouse	15	สำเร็จ
4	Customer.csv	dim_customer	200	สำเร็จ
5	Harvest.csv	fact_harvest	50,000	สำเร็จ
6	Sales.csv	fact_sales	30,000	สำเร็จ
7	Shipment.csv	fact_shipment	25,000	สำเร็จ
8	Inventory.csv	fact_inventory	64,974	สำเร็จ
	รวม		170,504	

7.3 ผลการทดสอบผู้ช่วย AI

ตารางที่ 17 แสดงผลลัพธ์การทดสอบด้วยคำถามภาษาไทย 3 ข้อผ่านหน้าผู้ช่วย AI ผลลัพธ์ที่ได้จากฐานข้อมูล RDS

คำถาม	ตารางที่ใช้	ผลลัพธ์	รูปแบบการแสดงผล
ยอดขายสุทธิทั้งหมดเท่าไร	fact_sales	2,958,202,618.86 บาท	ข้อความสรุป
ยอดขายรวมแยกตามพืช 5 อันดับแรก	fact_sales, dim_crop	1) กาแฟ 916,831,465.06 2) ทุเรียน 522,228,574.62 3) มังคุด 233,720,481.82 4) ลิ้นจี่ 227,084,772.53 5) ข้าวหอมมะลิ 221,326,490.21 (บาท)	ข้อความ + กราฟแท่ง
ยอดรับซื้อผลผลิตรวมแยกตามปี	fact_harvest, dim_date	ปี 2021 : 1,587,294,395.78 ปี 2022 : 1,553,500,220.03 ปี 2023 : 1,549,303,223.34 ปี 2024 : 1,518,597,712.34 ปี 2025 : 1,547,229,290.78 (บาท)	ข้อความ + กราฟเส้น

7.4 ผลการทดสอบความพร้อมใช้งาน

- **การตรวจสุขภาพ :** ตรวจสอบ /healthz ผ่าน ALB ได้ผล {"status":"ok","ai":"ok"} ทั้งเว็บ Express และ AI Agent FastAPI พร้อมทำงาน

- **การกระจายข้าม AZ :** เมื่อเพิ่มจำนวนเครื่องเป็น 2 เครื่อง ALB ลงทะเบียนเป้าหมายใหม่และเปลี่ยนสถานะเป็น healthy ได้หลังเครื่องติดตั้งเสร็จ (ประมาณ 5–8 นาที) เครื่องทั้งสองอยู่คนละ Availability Zone (us-east-1a และ us-east-1b)
- **การปรับขนาดอัตโนมัติ :** นโยบาย Target Tracking ลดจำนวนเครื่องอัตโนมัติเมื่อ CPU ต่ำกว่า เป้าหมาย และ ALB ตัดเครื่องที่ถูกยกเลิกออกจาก Target Group โดยไม่กระทบการ ใช้งาน
- **การหยุดและเปิดระบบ :** หลังปิดและเปิดระบบใหม่ (หยุด RDS และตั้ง Auto Scaling เป็น 0 แล้วเปิดกลับ) ข้อมูลที่โหลดไว้ยังอยู่ครบ และผู้ใช้กลับมาล็อกอินและสอบถาม AI ได้ตามปกติ เนื่องจากข้อมูลอยู่ใน RDS, S3 และ DynamoDB ซึ่งแยกจากเครื่อง EC2

7.5 ข้อจำกัดของระบบ

ตารางที่ 18 แสดงข้อจำกัดของระบบ

ที่	สิ่งที่วางแผนไว้	สิ่งที่ทำจริง	เหตุผล	ผลกระทบ
1	คลังข้อมูลบน Redshift (ตามสถาปัตยกรรม)	Amazon RDS for PostgreSQL ใน Private Subnet	Learner Lab ไม่อนุญาตให้ใช้ Amazon Redshift (ในบัญชีมีฟังก์ชันของ Lab ที่ปิด Redshift อัตโนมัติ) และ RDS PostgreSQL ตรงกับ ขอบเขตที่ระบุในรายงาน เสนอหัวข้อ	รองรับข้อมูลขนาดหลักแสนแถวได้ดี แต่ไม่ใช่ฐานข้อมูลแบบคอลัมน์ ประสิทธิภาพกับข้อมูลระดับหลายร้อยล้านแถว จะจำกัดกว่า Redshift
2	RDS แบบ Multi-AZ เพื่อพร้อมใช้งานสูง	RDS แบบ Single-AZ (DBMultiAZ = false) โดย DB Subnet Group ครอบคลุม 2 AZ	Multi-AZ ทำให้ต้นทุนเพิ่มเป็นสองเท่าและมักใช้ไม่ได้กับรุ่นทดลอง/free tier ของบัญชี Lab จึงปิดไว้ตามค่าเริ่มต้นของเทมเพลต	ไม่มี automatic failover ของฐานข้อมูล หาก AZ ที่ RDS อยู่ขัดข้อง ระบบวิเคราะห์จะใช้งานไม่ได้จนกว่าจะกู้คืน
3	เข้ารหัสเชื่อมต่อด้วย HTTPS	ALB รับเฉพาะ HTTP พอร์ต 80	ต้องมีชื่อโดเมนและใบรับรองจาก AWS Certificate Manager ซึ่งอยู่นอกขอบเขตและงบประมาณของ Lab	ข้อมูลระหว่างเบราว์เซอร์กับ ALB ไม่ถูกเข้ารหัส เหมาะกับต้นแบบ ไม่เหมาะกับใช้งานจริง (RDS และ Lambda ใช้ SSL)
4	เครือข่ายที่ปลอดภัยสูงสุด (เครื่อง EC2 อยู่ Private	เครื่อง EC2 อยู่ใน Public Subnet และไม่มี NAT Gateway	NAT Gateway มีค่าใช้จ่าย รายชั่วโมงสูง และเครื่อง EC2 ต้องเรียก API ของ Groq บนอินเทอร์เน็ต จึงให้มี Public IP แต่ปิดทุกพอร์ต ยกเว้นพอร์ต 3000 จาก ALB	เครื่อง EC2 มีที่อยู่ Public IP แม้ถูกจำกัดด้วย Security Group

ที่	สิ่งที่วางแผนไว้	สิ่งที่ทำจริง	เหตุผล	ผลกระทบ
	Subnet ผ่าน NAT Gateway)			
5	สร้าง IAM Role เฉพาะงานตามหลักสิทธิ์ขั้นต่ำ	ใช้ LabRole และ LabInstance Profile ที่ Lab จัดให้ทุกบริการ	บัญชี Learner Lab ไม่อนุญาตให้สร้าง IAM Role ใหม่	สิทธิ์ของ EC2, Lambda และ Step Functions กว้างกว่าที่จำเป็น
6	S3 เรียก Step Functions โดยตรงตามลูกศร Event Trigger ในแผนภาพ	S3 ส่ง event ไป EventBridge แล้ว Event Bridge เรียก Step Functions	S3 ไม่สามารถเริ่ม State Machine ได้โดยตรง ต้องมีตัวกลาง	มีส่วนประกอบเพิ่มแต่ผลลัพธ์การทำงานเหมือนที่ ออกแบบ
7	ETL ทำ SCD Type 2 เต็มรูปแบบ	เพิ่มแถวใหม่ valid_from, valid_to และ is_current แต่ไม่ปิด valid_to แถวเวอร์ชันเก่า และโหลดแบบ append เท่านั้น	ข้อมูลตัวอย่างที่ทดสอบโหลดครั้งแรก ไม่มีข้อมูล เปลี่ยนแปลงย้อนหลัง	การอัปโหลดไฟล์มิติเดิมซ้ำจะทำให้ได้แถวซ้ำ และไฟล์ข้อเท็จจริงเดิมซ้ำจะถูกปฏิเสธเพราะคีย์ซ้ำ

7.6 ข้อจำกัดอื่น ๆ ของระบบ

- **การพึ่งพาบริการภายนอก :** ผู้ช่วย AI ใช้บริการ Groq ผ่านอินเทอร์เน็ต หากบริการขัดข้องหรือ API key หมดสิทธิ์ ระบบจะตอบคำถามเชิงวิเคราะห์ไม่ได้ แต่ส่วนล็อกอินและนำเข้าข้อมูลยังใช้งานได้
- **ความเป็นส่วนตัวของข้อมูล :** คำถามของผู้ใช้และ schema ถูกส่งไปยัง LLM เพื่อสร้าง SQL และสรุปผล โดยจำกัดผลลัพธ์ไม่เกิน 100 แถว และตัดข้อมูลระดับบุคคลออกตั้งแต่ขั้น ETL
- **ความถูกต้องของ Text-to-SQL :** โมเดลอาจสร้าง SQL ไม่ตรงเจตนา แม้มี Guardrail ผู้ใช้จึงควรตรวจสอบ SQL ก่อนใช้ตัวเลขประกอบการตัดสินใจ
- **การจัดการผู้ใช้และสิทธิ์ :** บัญชีผู้ใช้และบทบาทถูกจัดเก็บใน DynamoDB โดยตรง ยังไม่มีหน้าเว็บสำหรับจัดการผู้ใช้หรือเปลี่ยนบทบาท
- **การปล่อยเวอร์ชันใหม่ :** เมื่ออัปเดตโค้ด เครื่อง EC2 เดิมไม่เปลี่ยนอัตโนมัติ ต้องยกเลิกเครื่องเดิมหรือสั่ง Instance Refresh เพื่อใช้โค้ดใหม่

ภาคผนวก (Appendices)

ภาคผนวก ก สคริปต์คำสั่ง SQL (DDL) สำหรับการสร้างตาราง

ก.1 การกำหนดพื้นที่จัดเก็บข้อมูล (Schema Initialization)

```
CREATE SCHEMA IF NOT EXISTS data_warehouse;

SET search_path TO data_warehouse, public;
```

ก.2 การสร้างตารางมิติ (Dimension Tables)

ก.2.1 dim_date (SCD Type 0: ห้ามแก้ไขหรือเปลี่ยนแปลงข้อมูลย้อนหลัง)

```
CREATE TABLE IF NOT EXISTS dim_date (
    date_key    INTEGER PRIMARY KEY,
    date        DATE NOT NULL,
    day         INTEGER NOT NULL,
    month       INTEGER NOT NULL,
    quarter     INTEGER NOT NULL,
    year        INTEGER NOT NULL,
    weekday     VARCHAR(15) NOT NULL
);
```

ก.2.2 dim_crop (SCD Type 1: เขียนทับข้อมูลล่าสุดเมื่อมีการเปลี่ยนแปลง)

```
CREATE TABLE IF NOT EXISTS dim_crop (
    crop_sk                 SERIAL PRIMARY KEY,
    crop_id                 VARCHAR(20) NOT NULL,
    crop_name               VARCHAR(100) NOT NULL,
    category                VARCHAR(50),
    unit                    VARCHAR(20),
```

```
    standard_price_per_unit    NUMERIC(12,2),
    season_months              VARCHAR(50),
    shelf_life_days            INTEGER
);
```

ก.2.3 dim_warehouse (SCD Type 3: เก็บประวัติชื่อคลังสินค้าเดิมและชื่อปัจจุบัน)

```
CREATE TABLE IF NOT EXISTS dim_warehouse (
    warehouse_sk               SERIAL PRIMARY KEY,
    warehouse_id               VARCHAR(20) NOT NULL,
    previous_warehouse_name    VARCHAR(150),
    current_warehouse_name     VARCHAR(150) NOT NULL,
    province                   VARCHAR(100),
    region                     VARCHAR(100),
    capacity_ton               INTEGER,
    manager_name               VARCHAR(150)
);
```

ก.2.4 dim_farmer (SCD Type 2: เก็บประวัติย้อนหลังทุกเวอร์ชันด้วย valid_from/to)

```
CREATE TABLE IF NOT EXISTS dim_farmer (
    farmer_sk                  SERIAL PRIMARY KEY,
    farmer_id                  VARCHAR(20) NOT NULL,
    farmer_name                VARCHAR(150) NOT NULL,
    province                   VARCHAR(100),
    region                     VARCHAR(100),
    cooperative                VARCHAR(150),
```

```
    farm_size_rai           NUMERIC(10,2),
    primary_crop            VARCHAR(100),
    valid_from              DATE NOT NULL,
    valid_to                DATE NOT NULL DEFAULT '9999-12-31',
    is_current              BOOLEAN NOT NULL DEFAULT TRUE
);
```

ก.2.5 dim_customer (SCD Type 2: เก็บประวัติย้อนหลังทุกเวอร์ชัน)

```
CREATE TABLE IF NOT EXISTS dim_customer (
    customer_sk         SERIAL PRIMARY KEY,
    customer_id         VARCHAR(20) NOT NULL,
    customer_name       VARCHAR(150) NOT NULL,
    customer_type       VARCHAR(50),
    province            VARCHAR(100),
    region              VARCHAR(100),
    credit_limit_thb    NUMERIC(14,2),
    valid_from          DATE NOT NULL,
    valid_to            DATE NOT NULL DEFAULT '9999-12-31',
    is_current          BOOLEAN NOT NULL DEFAULT TRUE
);
```

ก.3 การสร้างตารางข้อเท็จจริง (Fact Tables)

ชุดคำสั่งสร้างตารางข้อเท็จจริงสำหรับจัดเก็บข้อมูล Metrics พร้อมกำหนด Foreign Key เพื่อเชื่อมโยงกับตารางมิติและรักษา Referential Integrity

ก.3.1 fact_harvest (ระดับความละเอียด: 1 รายการรับซื้อผลผลิต)

```
CREATE TABLE IF NOT EXISTS fact_harvest (
```

```
    harvest_id          VARCHAR(20) PRIMARY KEY,
    harvest_date_key    INTEGER NOT NULL
        REFERENCES dim_date(date_key),
    farmer_sk           INTEGER NOT NULL
        REFERENCES dim_farmer(farmer_sk),
    crop_sk             INTEGER NOT NULL
        REFERENCES dim_crop(crop_sk),
    warehouse_sk        INTEGER NOT NULL
        REFERENCES dim_warehouse(warehouse_sk),
    quantity_kg         NUMERIC(14,2) NOT NULL,
    price_per_kg        NUMERIC(12,2) NOT NULL,
    total_amount_thb    NUMERIC(16,2) NOT NULL,
    quality_grade       VARCHAR(5)
);
```

ก.3.2 fact_sales (ระดับความละเอียด: 1 รายการขาย)

```
CREATE TABLE IF NOT EXISTS fact_sales (
    sales_id            VARCHAR(20) PRIMARY KEY,
    sale_date_key       INTEGER NOT NULL
        REFERENCES dim_date(date_key),
    customer_sk         INTEGER NOT NULL
        REFERENCES dim_customer(customer_sk),
    crop_sk             INTEGER NOT NULL
        REFERENCES dim_crop(crop_sk),
```

```
    warehouse_sk        INTEGER NOT NULL
        REFERENCES dim_warehouse(warehouse_sk),
    quantity_kg         NUMERIC(14,2) NOT NULL,
    unit_price_thb      NUMERIC(12,2) NOT NULL,
    total_amount_thb    NUMERIC(16,2) NOT NULL,
    discount_pct        NUMERIC(5,2),
    sale_channel        VARCHAR(50)
);
```

ก.3.3 fact_shipment (ระดับความละเอียด: 1 รอบการจัดส่ง)

```
CREATE TABLE IF NOT EXISTS fact_shipment (
    shipment_id             VARCHAR(20) PRIMARY KEY,
    shipment_date_key       INTEGER NOT NULL
        REFERENCES dim_date(date_key),
    delivery_date_key       INTEGER NOT NULL
        REFERENCES dim_date(date_key),
    customer_sk             INTEGER NOT NULL
        REFERENCES dim_customer(customer_sk),
    crop_sk                 INTEGER NOT NULL
        REFERENCES dim_crop(crop_sk),
    warehouse_sk            INTEGER NOT NULL
        REFERENCES dim_warehouse(warehouse_sk),
    total_weight_kg         NUMERIC(14,2) NOT NULL,
    shipping_cost_thb       NUMERIC(14,2),
```

```
    status                  VARCHAR(50),
    transport_mode          VARCHAR(50)
);
```

ก.3.4 fact_inventory (ระดับความละเอียด: สรุปยอดคงเหลือประจำวันต่อคลัง)

```
CREATE TABLE IF NOT EXISTS fact_inventory (
    inventory_id            VARCHAR(20) PRIMARY KEY,
    snapshot_date_key       INTEGER NOT NULL
        REFERENCES dim_date(date_key),
    warehouse_sk            INTEGER NOT NULL
        REFERENCES dim_warehouse(warehouse_sk),
    crop_sk                 INTEGER NOT NULL
        REFERENCES dim_crop(crop_sk),
    beginning_stock_kg      NUMERIC(14,2) NOT NULL,
    received_kg             NUMERIC(14,2) NOT NULL,
    sold_kg                 NUMERIC(14,2) NOT NULL,
    ending_stock_kg         NUMERIC(14,2) NOT NULL,
    unit_cost_thb           NUMERIC(12,2)
);
```

ก.4 การสร้างดัชนี (Index Creation)

สร้าง Index สำหรับข้อมูลการขายและการรับซื้อ

```
CREATE INDEX IF NOT EXISTS idx_fact_sales_date
ON fact_sales(sale_date_key);
CREATE INDEX IF NOT EXISTS idx_fact_sales_customer
```

```
ON fact_sales(customer_sk);

CREATE INDEX IF NOT EXISTS idx_fact_sales_crop

ON fact_sales(crop_sk);

CREATE INDEX IF NOT EXISTS idx_fact_sales_warehouse

ON fact_sales(warehouse_sk);

CREATE INDEX IF NOT EXISTS idx_fact_harvest_date

ON fact_harvest(harvest_date_key);
```

ก.4.1 สร้าง Index สำหรับสินค้าคงคลัง (ค้นหาตามช่วงเวลา)

```
CREATE INDEX IF NOT EXISTS idx_fact_inventory_date

ON fact_inventory(snapshot_date_key);
```

ก.4.2 สร้าง Index สำหรับข้อมูลลูกค้าและเกษตรกร (เพื่อค้นหาเฉพาะเรคคอร์ดปัจจุบัน)

```
CREATE INDEX IF NOT EXISTS idx_dim_customer_current

ON dim_customer(customer_id, is_current);

CREATE INDEX IF NOT EXISTS idx_dim_farmer_current

ON dim_farmer(farmer_id, is_current);
```

ก.5 ตัวอย่างสคริปต์กระบวนการ ETL บน AWS Lambda (Python)

สคริปต์ Python สำหรับ AWS Lambda เพื่อทำ ETL ไฟล์ Sales.csv จาก S3 เข้าสู่ fact_sales บน Amazon RDS พร้อมทำความสะอาดข้อมูล, Surrogate Key Lookup และ Bulk Upsert

```
import csv

import io

import json

import logging

import os

import urllib.parse
```

```
from datetime import datetime

import boto3

import psycopg2

from psycopg2.extras import execute_batch

logger = logging.getLogger()

logger.setLevel(logging.INFO)

s3_client = boto3.client("s3")

# ดึงค่าการเชื่อมต่อฐานข้อมูลจาก Lambda Environment Variables

DB_HOST = os.environ.get("DB_HOST", "agri-dwh-dbmaster.csszwkbhk6ok.us-east-
1.rds.amazonaws.com")

DB_PORT = int(os.environ.get("DB_PORT", 5432))

DB_NAME = os.environ.get("DB_NAME", "postgres")

DB_USER = os.environ.get("DB_USER", "admin_agri")

DB_PASSWORD = os.environ.get("DB_PASSWORD", "")

DB_SCHEMA = os.environ.get("DB_SCHEMA", "public")

def get_db_connection():

    """เชื่อมต่อไปยัง Amazon RDS PostgreSQL"""

    return psycopg2.connect(

        host=DB_HOST,

        port=DB_PORT,

        dbname=DB_NAME,

        user=DB_USER,

        password=DB_PASSWORD,
```

```
        connect_timeout=10,
        options=f"-c search_path={DB_SCHEMA},public"
    )
def parse_date_key(date_str: str) -> int:
    """แปลงวันที่ 'YYYY-MM-DD' เป็น integer date_key YYYYMMDD สำหรับ Star Schema"""
    if not date_str:
        return 19000101
    clean_date = date_str.strip().split(" ")[0]
    dt = datetime.strptime(clean_date, "%Y-%m-%d")
    return int(dt.strftime("%Y%m%d"))
def clean_float(value, default=0.0) -> float:
    """แปลงตัวเลขอย่างปลอดภัย รองรับค่าว่างและเครื่องหมายจุลภาค"""
    if value is None or str(value).strip() == "":
        return default
    try:
        return float(str(value).replace(",", "").strip())
    except (ValueError, TypeError):
        return default
def fetch_dimension_lookups(cursor) -> dict:
    """ดึงแมปปิ้ง Business Key -> Surrogate Key เก็บไว้ใน Memory เพื่อทำ Fast Lookup"""
    lookups = {"customer": {}, "crop": {}, "warehouse": {}}
    cursor.execute("SELECT customer_id, customer_sk FROM dim_customer WHERE is_current = TRUE;")
    for cid, csk in cursor.fetchall():
```

```
        lookups["customer"][cid] = csk
    cursor.execute("SELECT crop_id, crop_sk FROM dim_crop;")
    for cid, csk in cursor.fetchall():
        lookups["crop"][cid] = csk
    cursor.execute("SELECT warehouse_id, warehouse_sk FROM dim_warehouse;")
    for wid, wsk in cursor.fetchall():
        lookups["warehouse"][wid] = wsk
    return lookups
def clean_sales_row(row: dict, lookups: dict) -> tuple | None:
    """ทำความสะอาด ตรวจสอบความถูกต้อง และแปลงข้อมูล 1 แถวของ Sales.csv"""
    sales_id = row.get("sales_id", "").strip()
    if not sales_id:
        return None  # ทิ้งแถวที่ไม่มี Primary Key
    sale_date = row.get("sale_date", "").strip()
    sale_date_key = parse_date_key(sale_date)
    customer_id = row.get("customer_id", "").strip()
    crop_id = row.get("crop_id", "").strip()
    warehouse_id = row.get("warehouse_id", "").strip()
    # ทำ Surrogate Key Lookup
    customer_sk = lookups["customer"].get(customer_id, 1)
    crop_sk = lookups["crop"].get(crop_id, 1)
    warehouse_sk = lookups["warehouse"].get(warehouse_id, 1)
    quantity_kg = clean_float(row.get("quantity_kg"))
```

```
    unit_price_thb = clean_float(row.get("unit_price_thb"))
    total_amount_thb = clean_float(row.get("total_amount_thb"))
    discount_pct = clean_float(row.get("discount_pct"), default=0.0)
    sale_channel = row.get("sale_channel", "ตรงจากคลัง").strip()
    # Data Quality Validation: ป้องกันค่าติดลบ
    if quantity_kg < 0 or unit_price_thb < 0:
        logger.warning(f"Discarding invalid row: {sales_id} (negative values)")
        return None
    # คำนวณยอดเงินใหม่ถ้าใน CSV เป็น 0 หรือเว้นว่าง
    if total_amount_thb <= 0 and quantity_kg > 0 and unit_price_thb > 0:
        total_amount_thb = round(quantity_kg * unit_price_thb * (1 - discount_pct / 100.0), 2)
    return (sales_id, sale_date_key, customer_sk, crop_sk, warehouse_sk, quantity_kg, unit_price_thb,
total_amount_thb, discount_pct, sale_channel)
def lambda_handler(event, context):
    """ฟังก์ชัน Entry Point ที่ AWS Lambda จะเรียกเมื่อมี S3 Upload Event"""
    logger.info("Received S3 Event: %s", json.dumps(event))
    records = event.get("Records", [])
    if not records:
        return {"statusCode": 200, "body": "No records in event"}
    total_inserted = 0
    for record in records:
        bucket = record["s3"]["bucket"]["name"]
        key = urllib.parse.unquote_plus(record["s3"]["object"]["key"], encoding="utf-8")
```

```
        logger.info(f"Extracting file from S3: s3://{bucket}/{key}")
        # 1. EXTRACT: อ่านไฟล์ CSV จาก S3 Bucket
        response = s3_client.get_object(Bucket=bucket, Key=key)
        csv_content = response["Body"].read().decode("utf-8-sig")
        csv_reader = csv.DictReader(io.StringIO(csv_content))
        # 2. TRANSFORM & CLEAN
        conn = get_db_connection()
        try:
            with conn.cursor() as cur:
                lookups = fetch_dimension_lookups(cur)
                cleaned_rows = []
                for row in csv_reader:
                    cleaned = clean_sales_row(row, lookups)
                    if cleaned:
                        cleaned_rows.append(cleaned)
                logger.info(f"Cleaned {len(cleaned_rows)} valid rows from {key}")
                # 3. LOAD: Bulk Upsert ลงตาราง fact_sales ใน RDS
                upsert_sql = """
                    INSERT INTO fact_sales (sales_id, sale_date_key, customer_sk, crop_sk, warehouse_sk,
quantity_kg, unit_price_thb, total_amount_thb, discount_pct, sale_channel) VALUES (%s, %s, %s, %s,
%s, %s, %s, %s, %s, %s)
                    ON CONFLICT (sales_id) DO UPDATE SET
                        sale_date_key = EXCLUDED.sale_date_key,
                        quantity_kg = EXCLUDED.quantity_kg,
```

```
                    unit_price_thb = EXCLUDED.unit_price_thb,
                    total_amount_thb = EXCLUDED.total_amount_thb,
                    discount_pct = EXCLUDED.discount_pct,
                    sale_channel = EXCLUDED.sale_channel;
            """
            execute_batch(cur, upsert_sql, cleaned_rows, page_size=1000)
            conn.commit()
            total_inserted += len(cleaned_rows)
            logger.info(f"Successfully loaded {len(cleaned_rows)} rows into RDS fact_sales")
    except Exception as exc:
        conn.rollback()
        logger.error(f"ETL Execution Failed: {exc}")
        raise exc
    finally:
        conn.close()
return {
    "statusCode": 200,
    "body": json.dumps({
        "message": "ETL job executed successfully",
        "total_rows_loaded": total_inserted
    }, ensure_ascii=False)
}
```

ภาคผนวก ข โค้ดโปรแกรม Python สำหรับ AWS Lambda ETL

Lambda ชื่อ agrichain-etl เลือกฟังก์ชัน ETL จากคำในชื่อไฟล์ ทำความสะอาดข้อมูล แปลงรหัส ธุรกิจเป็น surrogate key และโหลดเข้า RDS

```
def get_surrogate_key_mapping(engine, table_name, sk_col, bk_col):
    query = f"SELECT {sk_col}, {bk_col} FROM {DB_SCHEMA}.{table_name}"
    if table_name in ("dim_farmer", "dim_customer"):
        query += " WHERE is_current = true"
    df_lookup = pd.read_sql(query, con=engine)
    return dict(zip(df_lookup[bk_col], df_lookup[sk_col]))
def load(df, engine, table):
    cols = pd.read_sql( text("SELECT column_name FROM information_schema.columns "
            "WHERE table_schema = :s AND table_name = :t"),
        con=engine, params={"s": DB_SCHEMA, "t": table},)["column_name"].tolist()
    dropped = [c for c in df.columns if c not in cols]
    if dropped:
        print(f"{table}: ignoring source-only columns {dropped}")
    df = df[[c for c in df.columns if c in cols]]
    df.to_sql(name=table, con=engine, schema=DB_SCHEMA, if_exists="append", index=False,
       method="multi",chunksize=500,)
    return table
def date_key(series):
    return series.astype(str).str.slice(0, 10).str.replace("-", "", regex=False).astype(int)
def map_keys(df, engine, specs):
    for table, sk_col, bk_col in specs:
        df[sk_col] = df[bk_col].map(get_surrogate_key_mapping(engine, table, sk_col, bk_col))
        missing = df[sk_col].isna()
        if missing.any():
            bad = sorted(df.loc[missing, bk_col].astype(str).unique())[:10]
            raise ValueError(
                f"{missing.sum()} row(s) reference {bk_col} not found in {table} "
                f"(e.g. {bad}). Upload the dimension files first.")
        df[sk_col] = df[sk_col].astype(int)
```

```
    return df
def etl_shipment(df, engine):
    df["shipment_date_key"] = date_key(df["shipment_date"])
    df["delivery_date"] = df["delivery_date"].fillna("9999-12-31")
    df["delivery_date_key"] = date_key(df["delivery_date"])
    df = map_keys(df, engine, [
        ("dim_customer", "customer_sk", "customer_id"),
        ("dim_crop", "crop_sk", "crop_id"),
        ("dim_warehouse", "warehouse_sk", "warehouse_id"),])
    df = df.drop(columns=[
        "customer_id", "crop_id", "warehouse_id", "shipment_date",
        "delivery_date", "driver_name", "vehicle_plate",])
    return load(df, engine, "fact_shipment")
def etl_inventory(df, engine):
    df["snapshot_date_key"] = date_key(df["snapshot_date"])
    df["received_kg"] = df["received_kg"].fillna(0.00)
    df["sold_kg"] = df["sold_kg"].fillna(0.00)
    df = map_keys(df, engine, [
        ("dim_crop", "crop_sk", "crop_id"),
        ("dim_warehouse", "warehouse_sk", "warehouse_id"), ])
    df = df.drop(columns=["crop_id", "warehouse_id", "snapshot_date", "stock_status"])
    return load(df, engine, "fact_inventory")
ROUTES = [ ("Crop", etl_crop), ("Farmer", etl_farmer),  ("Warehouse", etl_warehouse),  ("Customer",
etl_customer),  ("Harvest", etl_harvest),  ("Sales", etl_sales), ("Shipment", etl_shipment), ("Inventory",
etl_inventory),]
def run_etl(event):
    bucket, key = extract_s3_object(event)
    print(f"ETL start: s3://{bucket}/{key}")
    if not key.lower().endswith(".csv"):
        print(f"Skipping non-CSV object: {key}")
        return {"statusCode": 200, "body": f"Skipped non-CSV object {key}"}
    filename = key.rsplit("/", 1)[-1]
```

```
    handler = next((fn for word, fn in ROUTES if word in filename), None)
    if handler is None:
        raise ValueError(f"File pattern not recognized: {key}")
    obj = s3_client.get_object(Bucket=bucket, Key=key)
    df = pd.read_csv(obj["Body"], encoding="utf-8-sig")  # source CSVs carry a BOM
    target_table = handler(df, get_db_engine())
    msg = f"ETL completed for {key}: inserted {len(df)} rows into {DB_SCHEMA}.{target_table}"
    print(msg)
    return {"statusCode": 200, "body": msg, "table": target_table, "rows": int(len(df))}
```

ภาคผนวก ค คู่มือการติดตั้งและการเปิด-ปิดระบบ

ค.1 การติดตั้งครั้งแรก ต้องมี Python 3, boto3 และรหัสเข้าถึงจากหน้า AWS Details ของ Learner Lab

```
python -m pip install boto3
export AWS_ACCESS_KEY_ID=...        # จากหน้า AWS Details ของ Learner Lab
export AWS_SECRET_ACCESS_KEY=...
export AWS_SESSION_TOKEN=...
export AWS_DEFAULT_REGION=us-east-1
python infra/deploy.py --groq-key <GROQ_API_KEY> \
      --admin-username exec01 --admin-name exec01 --admin-password "<รหัสผ่าน>"
```

ค.2 การปิดระบบชั่วคราว

```
aws autoscaling update-auto-scaling-group --auto-scaling-group-name agrichain-asg --min-size 0 --desired-capacity 0
aws rds stop-db-instance --db-instance-identifier agrichain-dwh-db
```

ค.3 การเปิดระบบกลับ

```
aws rds start-db-instance --db-instance-identifier agrichain-dwh-db        # รอสถานะ available
aws autoscaling update-auto-scaling-group --auto-scaling-group-name agrichain-asg --min-size 1 --desired-capacity 1
```

ค.4 การปล่อยโค้ดเวอร์ชันใหม่

รันคำสั่งติดตั้งเดิมซ้ำ (อัปเดต stack เดิม) จากนั้นต้องเปลี่ยนเครื่อง EC2 ให้ใช้โค้ดใหม่ โดย เพิ่มจำนวนเครื่องแล้วยกเลิกเครื่องเก่าทีละตัว (คำสั่ง Instance Refresh ถูกจำกัดสิทธิ์ใน Learner Lab)

ภาคผนวก ง Data Dictionary ของแหล่งข้อมูลต้นทาง

ตารางที่ 19 Data Dictionary : Harvest (ข้อมูลการรับซื้อผลผลิต)

Column	Description	Example
harvest_id	รหัสอ้างอิงการรับซื้อผลผลิต	HRV-000001
farmer_id	รหัสเกษตรกรผู้นำผลผลิตมาขาย	FRM-0028
crop_id	รหัสผลผลิตที่รับซื้อ	CRP-004
warehouse_id	รหัสคลังสินค้าที่รับเข้า	WH-006
harvest_date	วันที่รับซื้อ	2025-04-28
quantity_kg	ปริมาณที่รับซื้อ (กิโลกรัม)	16732.67
price_per_kg	ราคารับซื้อต่อกิโลกรัม (บาท)	8.15
total_amount_thb	ยอดรวมเงินที่จ่ายให้เกษตรกร (บาท)	136371.26
quality_grade	เกรดคุณภาพผลผลิต (A, B, C)	C
inspector_note	หมายเหตุจากการตรวจสอบคุณภาพ	ผ่านการตรวจสอบ

ตารางที่ 20 Data Dictionary : Sales (ข้อมูลการขาย)

Column	Description	Example
sales_id	รหัสอ้างอิงรายการขาย	SAL-000001
order_id	รหัสคำสั่งซื้อ	ORD-000001
customer_id	รหัสลูกค้าที่ซื้อสินค้า	CUS-0079
crop_id	รหัสผลผลิต/สินค้าที่ขาย	CRP-014
warehouse_id	รหัสคลังสินค้าที่ตัดจ่าย	WH-007

Column	Description	Example
sale_date	วันที่บันทึกการขาย	2022-09-05
quantity_kg	ปริมาณที่ขาย (กิโลกรัม)	5829.68
unit_price_thb	ราคาขายต่อกิโลกรัม (บาท)	120.06
total_amount_thb	ยอดเงินรวมก่อนหักส่วนลด (บาท)	699911.38
discount_pct	เปอร์เซ็นต์ส่วนลด (%)	10
payment_status	สถานะการชำระเงิน	ชำระแล้ว
sale_channel	ช่องทางการขาย	ออนไลน์

ตารางที่ 21 Data Dictionary : Shipment (ข้อมูลการจัดส่ง)

Column	Description	Example
shipment_id	รหัสอ้างอิงรอบการจัดส่ง	SHP-000001
warehouse_id	รหัสคลังสินค้าต้นทาง	WH-007
customer_id	รหัสลูกค้าระปลายทาง	CUS-0080
crop_id	รหัสสินค้าที่จัดส่ง	CRP-012
shipment_date	วันที่เริ่มจัดส่ง	2022-08-28
delivery_date	วันที่ส่งมอบสำเร็จ	2022-09-03
status	สถานะการจัดส่ง	จัดส่งแล้ว
total_weight_kg	น้ำหนักรวมของการจัดส่ง (กิโลกรัม)	5041.34
shipping_cost_thb	ค่าใช้จ่ายในการจัดส่ง (บาท)	8356.45

Column	Description	Example
transport_mode	รูปแบบการขนส่ง (รถยนต์, เรือ)	เรือ
driver_name	ชื่อพนักงานขับรถ/ผู้รับผิดชอบ	ประสิทธิ์ โชคดี
vehicle_plate	ทะเบียนรถ/รหัสพาหนะ	ขข-5703 ชม

ตารางที่ 22 Data Dictionary : Inventory (ข้อมูลสินค้าคงคลัง)

Column	Description	Example
inventory_id	รหัสอ้างอิงบันทึกคลังสินค้า	INV-000001
warehouse_id	รหัสคลังสินค้า	WH-001
crop_id	รหัสสินค้า	CRP-013
snapshot_date	วันที่บันทึกยอดคงเหลือ	2021-01-01
beginning_stock_kg	ยอดสต็อกยกมาตอนต้นรอบ (กิโลกรัม)	27893.61
received_kg	ปริมาณที่รับเข้าคลัง (กิโลกรัม)	9129.18
sold_kg	ปริมาณที่เบิก/ขายออก (กิโลกรัม)	5501.64
ending_stock_kg	ยอดสต็อกคงเหลือสิ้นรอบ (กิโลกรัม)	31521.15
unit_cost_thb	ต้นทุนเฉลี่ยต่อหน่วย (บาท)	8.2
stock_status	สถานะปริมาณสต็อกคุณภาพ (สูง/ต่ำ)	สูง

ตารางที่ 23 Data Dictionary : Farmer (ข้อมูลเกษตรกร)

Column	Description	Example
farmer_id	รหัสประจำตัวเกษตรกร	FRM-0001
farmer_name	ชื่อ-นามสกุลเกษตรกร	ณรงค์ พิมพา
national_id	รหัสประจำตัวประชาชน	3689120466741
province	จังหวัดภูมิลำเนา	ระยอง
region	ภูมิภาค	ภาคตะวันออก
district	อำเภอ	อ.บ้านโฮ่ง
cooperative	ชื่อสหกรณ์ต้นสังกัด	สหกรณ์ชาวสวนปาล์มชุมพร จำกัด
farm_size_rai	ขนาดพื้นที่เพาะปลูก (ไร่)	121.30
primary_crop	ผลผลิตหลักที่เพาะปลูก	ปาล์มน้ำมัน
registration_date	วันที่ขึ้นทะเบียน	2020-04-05
status	สถานะการเป็นสมาชิก	ใช้งาน
phone	เบอร์โทรศัพท์ติดต่อ	925641184

ตารางที่ 24 Data Dictionary : Customer (ข้อมูลลูกค้า)

Column	Description	Example
customer_id	รหัสลูกค้า/คู่ค้า	CUS-0001
customer_name	ชื่อบริษัทหรือชื่อลูกค้า	บริษัท ใต้ดีผลไม้ จำกัด
customer_type	ประเภทลูกค้า (ห้างค้าปลีก, สหกรณ์)	ห้างค้าปลีก
province	จังหวัดที่ตั้งองค์กรลูกค้า	หนองคาย
region	ภูมิภาค	ภาคตะวันออกเฉียงเหนือ
contact_person	ชื่อผู้ติดต่อ	ยุทธนา โชคดี
phone	เบอร์โทรศัพท์ติดต่อ	972175697
email	อีเมลลูกค้า	contact1@agri-buyer.co.th
credit_limit_thb	วงเงินเครดิตที่อนุมัติ (บาท)	500000
is_active	สถานะการดำเนินธุรกิจกับสหกรณ์	Y

ตารางที่ 25 Data Dictionary : Crop (ข้อมูลผลผลิต)

Column	Description	Example
crop_id	รหัสผลผลิตทางการเกษตร	CRP-001
crop_name	ชื่อผลผลิต	ข้าวนาปรัง
category	หมวดหมู่ผลผลิต (ธัญพืช, ผลไม้ ฯลฯ)	ธัญพืช
unit	หน่วยบรรจุภัณฑ์	กิโลกรัม
standard_price_per_unit	ราคามาตรฐานอ้างอิงต่อหน่วย (บาท)	11.20

Column	Description	Example
season_months	เดือนที่เป็นฤดูกาลเก็บเกี่ยว (รายเดือน)	1, 2, 3, 4, 5
shelf_life_days	อายุการเก็บรักษาสูงสุด (วัน)	7
is_active	สถานะการรับซื้อ/ขาย	Y

ตารางที่ 26 Data Dictionary : Warehouse (ข้อมูลคลังสินค้า)

Column	Description	Example
warehouse_id	รหัสคลังสินค้า/ไซโล	WH-001
warehouse_name	ชื่อคลังสินค้า	คลังสินค้าเชียงใหม่
province	จังหวัดที่ตั้งคลังสินค้า	เชียงใหม่
region	ภูมิภาค	ภาคเหนือ
address	ที่อยู่แบบเต็ม	เลขที่ 160 ถ.เกษตรกลาง เชียงใหม่
capacity_ton	ความจุสูงสุดของคลัง (ตัน)	1500
current_utilization_pct	เปอร์เซ็นต์พื้นที่ใช้งานปัจจุบัน (%)	49.69
manager_name	ชื่อผู้จัดการคลังสินค้า	รัตนา ศรีสุข
contact_phone	เบอร์โทรศัพท์คลังสินค้า	824896273
is_active	สถานะการเปิดใช้งานคลัง	Y

ภาคผนวก จ ตาราง Dimension และ Fact

ตารางที่ 27 แสดงตาราง dim_crop (ข้อมูลมิติผลผลิต)

Field Name	Data Type	Description	Example
crop_sk	Integer	รหัส Surrogate Key ของผลผลิต (Primary Key)	1
crop_id	Varchar	รหัสผลผลิตจากระบบต้นทาง	CRP-001
crop_name	Varchar	ชื่อผลผลิต	ข้าวนาปรัง
category	Varchar	หมวดหมู่ผลผลิต	ธัญพืช
unit	Varchar	หน่วยบรรจุภัณฑ์	กิโลกรัม
standard_price_per_unit	Numeric	ราคามาตรฐานอ้างอิงต่อหน่วย	11.20
season_months	Varchar	เดือนที่เป็นฤดูกาลเก็บเกี่ยว	1, 2, 3, 4, 5
shelf_life_days	Integer	อายุการเก็บรักษาสูงสุด (วัน)	7

ตารางที่ 28 แสดงตาราง dim_warehouse (ข้อมูลมิติคลังสินค้า)

Field Name	Data Type	Description	Example
warehouse_sk	Integer	รหัส Surrogate Key ของคลังสินค้า (Primary Key)	1
warehouse_id	Varchar	รหัสคลังสินค้าจากระบบต้นทาง	WH-001
previous_warehouse_name	Varchar	ชื่อคลังสินค้าก่อนหน้า (รองรับ SCD Type 3)	[null]
current_warehouse_name	Varchar	ชื่อคลังสินค้าปัจจุบัน	คลังสินค้าเชียงใหม่

Field Name	Data Type	Description	Example
province	Varchar	จังหวัดที่ตั้งคลังสินค้า	เชียงใหม่
region	Varchar	ภูมิภาค	ภาคเหนือ
capacity_ton	Integer	ความจุสูงสุดของคลัง (ตัน)	1500
manager_name	Varchar	ชื่อผู้จัดการคลังสินค้า	รัตนา ศรีสุข

ตารางที่ 29 แสดงตาราง dim_farmer (ข้อมูลมิติเกษตรกร)

Field Name	Data Type	Description	Example
farmer_sk	Integer	รหัส Surrogate Key ของเกษตรกร (Primary Key)	1
farmer_id	Varchar	รหัสประจำตัวเกษตรกรจากระบบต้นทาง	FRM-0001
farmer_name	Varchar	ชื่อ-นามสกุลเกษตรกร	ณรงค์ พิมพา
cooperative	Varchar	ชื่อสหกรณ์ต้นสังกัด	สหกรณ์ชาวสวนปาล์มชุมพร จำกัด
farm_size_rai	Numeric	ขนาดพื้นที่เพาะปลูก (ไร่)	121.30
primary_crop	Varchar	ผลผลิตหลักที่เพาะปลูก	ปาล์มน้ำมัน
valid_from	Date	วันที่เริ่มต้นการใช้ข้อมูลแถวนี้ (รองรับ SCD Type 2)	2020-04-05
valid_to	Date	วันที่สิ้นสุดการใช้ข้อมูลแถวนี้	9999-12-31

Field Name	Data Type	Description	Example
is_current	Boolean	ระบุว่าเป็นข้อมูลที่ใช้งานในปัจจุบันหรือไม่	true
province	Varchar		
region	Varchar		

ตารางที่ 30 แสดงตาราง dim_customer (ข้อมูลมิติลูกค้าและคู่ค้า)

Field Name	Data Type	Description	Example
customer_sk	Integer	รหัส Surrogate Key ของลูกค้า (Primary Key)	1
customer_id	Varchar	รหัสลูกค้า/คู่ค้าจากระบบต้นทาง	CUS-0001
customer_name	Varchar	ชื่อบริษัทหรือชื่อลูกค้า	บริษัท ใต้ดี ผลไม้ จำกัด
customer_type	Varchar	ประเภทลูกค้า (เช่น ห้างค้าปลีก, สหกรณ์)	ห้างค้าปลีก
region	Varchar	ภูมิภาค	ภาคตะวันออกเฉียงเหนือ
credit_limit_thb	Numeric	วงเงินเครดิตที่อนุมัติ (บาท)	500000.00
valid_from	Date	วันที่เริ่มต้นการใช้ข้อมูลแถวนี้ (รองรับ SCD Type 2)	2022-01-01
valid_to	Date	วันที่สิ้นสุดการใช้ข้อมูลแถวนี้	9999-12-31

Field Name	Data Type	Description	Example
is_current	Boolean	ระบุว่าเป็นข้อมูลที่ใช้งานในปัจจุบันหรือไม่	true
province	Varchar		

ตารางที่ 31 แสดงตาราง dim_date (ข้อมูลมิติเวลา)

Field Name	Data Type	Description	Example
date_key	Integer	รหัส Surrogate Key ของวันที่ (รูปแบบ YYYYMMDD)	20250428
date	Date	วันที่เต็ม	2025-04-28
day	Integer	วันที่ในเดือน	28
month	Integer	เดือน (1-12)	4
quarter	Integer	ไตรมาส (1-4)	2
year	Integer	ปี (YYYY)	2025
weekday	Varchar	ชื่อวันในสัปดาห์	Monday

ตารางที่ 32 แสดงตาราง fact_harvest (ข้อมูลข้อเท็จจริงการรับซื้อ)

Field Name	Data Type	Description	Example
harvest_id	Varchar	รหัสรายการรับซื้อ (Primary Key)	HRV-000001
harvest_date_key	Integer	คีย์วันที่รับซื้อ (Foreign Key เชื่อม dim_date)	20250428
farmer_sk	Integer	คีย์เกษตรกร (Foreign Key เชื่อม dim_farmer)	28

Field Name	Data Type	Description	Example
crop_sk	Integer	คีย์ผลผลิต (Foreign Key เชื่อม dim_crop)	4
warehouse_sk	Integer	คีย์คลังสินค้า (Foreign Key เชื่อม dim_warehouse)	6
quantity_kg	Numeric	ปริมาณที่รับซื้อ (กิโลกรัม)	16732.67
price_per_kg	Numeric	ราคารับซื้อต่อกิโลกรัม (บาท)	8.15
total_amount_thb	Numeric	ยอดเงินรวมที่จ่ายให้เกษตรกร (บาท)	136371.26
quality_grade	Varchar	เกรดคุณภาพผลผลิต (A, B, C)	C

ตารางที่ 33 แสดงตาราง fact_sales (ข้อมูลข้อเท็จจริงการขาย)

Field Name	Data Type	Description	Example
sales_id	Varchar	รหัสรายการขาย (Primary Key)	SAL-000001
sale_date_key	Integer	คีย์วันที่ขาย (Foreign Key เชื่อม dim_date)	20220905
customer_sk	Integer	คีย์ลูกค้า (Foreign Key เชื่อม dim_customer)	79
crop_sk	Integer	คีย์ผลผลิต (Foreign Key เชื่อม dim_crop)	14
warehouse_sk	Integer	คีย์คลังสินค้า (Foreign Key เชื่อม dim_warehouse)	7
quantity_kg	Numeric	ปริมาณที่ขาย (กิโลกรัม)	5829.68
unit_price_thb	Numeric	ราคาขายต่อหน่วย (บาท)	120.06

Field Name	Data Type	Description	Example
total_amount_thb	Numeric	ยอดขายรวม (บาท)	699911.38
discount_pct	Numeric	ส่วนลด (%)	10.00
sale_channel	Varchar	ช่องทางการขาย	ออนไลน์

ตารางที่ 34 แสดงตาราง fact_shipment (ข้อมูลข้อเท็จจริงการจัดส่ง)

Field Name	Data Type	Description	Example
shipment_id	Varchar	รหัสรายการจัดส่ง (Primary Key)	SHP-000001
shipment_date_key	Integer	คีย์วันที่เริ่มจัดส่ง (Foreign Key เชื่อม dim_date)	20220828
delivery_date_key	Integer	คีย์วันที่ส่งมอบสำเร็จ (Foreign Key เชื่อม dim_date)	20220903
customer_sk	Integer	คีย์ลูกค้า (Foreign Key เชื่อม dim_customer)	80
crop_sk	Integer	คีย์สินค้าที่จัดส่ง (Foreign Key เชื่อม dim_crop)	12
warehouse_sk	Integer	คีย์คลังสินค้าต้นทาง (Foreign Key เชื่อม dim_warehouse)	7
total_weight_kg	Numeric	น้ำหนักรวมของการจัดส่ง (กิโลกรัม)	5041.34
shipping_cost_thb	Numeric	ค่าใช้จ่ายในการจัดส่ง (บาท)	8356.45
status	Varchar	สถานะการจัดส่ง	จัดส่งแล้ว
transport_mode	Varchar	รูปแบบการขนส่ง (รถยนต์, เรือ)	เรือ

ตารางที่ 35 แสดงตาราง fact_inventory (ข้อมูลข้อเท็จจริงสินค้าคงคลัง)

Field Name	Data Type	Description	Example
inventory_id	Varchar	รหัสอ้างอิงบันทึกคลังสินค้า (Primary Key)	INV-000001
snapshot_date_key	Integer	คีย์วันที่บันทึกยอด (Foreign Key เชื่อม dim_date)	20210101
warehouse_sk	Integer	คีย์คลังสินค้า (Foreign Key เชื่อม dim_warehouse)	1
crop_sk	Integer	คีย์สินค้า (Foreign Key เชื่อม dim_crop)	13
beginning_stock_kg	Numeric	ยอดสต็อกยกมาตอนต้นรอบ (กิโลกรัม)	27893.61
received_kg	Numeric	ปริมาณที่รับเข้าคลัง (กิโลกรัม)	9129.18
sold_kg	Numeric	ปริมาณที่เบิก/ขายออก (กิโลกรัม)	5501.64
ending_stock_kg	Numeric	ยอดสต็อกคงเหลือสิ้นรอบ (กิโลกรัม)	31521.15
unit_cost_thb	Numeric	ต้นทุนเฉลี่ยต่อหน่วย (บาท)	8.2